Example-Driven Intent Synthesis for Constrained Data Bundle Retrieval: Focused Text Snippet Extraction and Beyond

Whanhee Cho
University of Utah
whanhee.cho@utah.edu

Kuangfei Long
Boston University
longkuangfei@outlook.com

Mahmood Jasim
Louisiana State University
mjasim@lsu.edu

Matteo Brucato
OSM Data
matteo@osm-data.com

Alexandra Meliou
UMass Amherst
ameli@cs.umass.edu

Peter J. Haas
UMass Amherst
phaas@cs.umass.edu

Anna Fariha
University of Utah
afariha@cs.utah.edu

ABSTRACT

Selecting a *bundle* of items that collectively satisfies constraints is a fundamental task across databases, recommender systems, and text summarization. Unlike traditional top-k retrieval, bundle retrieval is inherently combinatorial and, in general, NP-hard. Although package queries can efficiently retrieve bundles given a well-formed query, two key user-centric challenges remain: (1) expressing and tuning multi-dimensional bundle intent through a user-friendly interface, and (2) ensuring feasibility when the query yields empty results. We introduce Ex2Bundle, an *Example-driven Bundle retrieval* framework that enables users to specify their intent through *example bundles* and automatically synthesizes package queries that capture the intent implicit in those examples via aggregate constraints. Ex2Bundle also addresses a challenge unique to bundle retrieval: when inferred constraints are infeasible over the target data, our *data-aware constraint relaxation* minimally adjusts the constraint bounds while preserving alignment with user intent. We instantiate a specific application of *focused text snippet extraction* to demonstrate the efficacy of Ex2Bundle. Extensive experiments over real-world datasets and a user study demonstrate that Ex2Bundle improves usability and consistently returns intent-aligned bundles even under distributional shifts of the target database.

Artifact Link: https://github.com/kuangfei-long/ex2bundle.

1 INTRODUCTION

Data *bundle* retrieval is a fundamental task across multiple domains: package queries retrieve packages (sets of tuples) from relational databases under aggregate constraints [11]; recommender systems suggest playlists (sets of songs) or combo offers (sets of products) tailored to user preferences [5]; and extractive text summarization systems retrieve subsets of sentences from documents that form coherent summaries based on user queries [20, 60, 85]. Unlike traditional retrieval that returns individual or top-k items, bundle retrieval involves selecting a *set* (or package [11]) whose elements must collectively optimize global objectives while satisfying set-level constraints that align with user preferences. Consequently, the inclusion or exclusion of any single item affects the feasibility and overall quality of the bundle, making the problem inherently combinatorial and NP-hard [11]. While systems exist for the efficient retrieval of data bundles from this combinatorial search space [11], two user-centric challenges remain as primary obstacles: (1) the *interface* challenge—how can humans easily express their intent of the desired data bundles—and (2) the *feasibility* challenge—how to ensure non-empty answers for the user queries.

Challenge 1: Usable interface. The first challenge is communicating the user intent of the desired data bundle. Like SQL, the Package Query Language (PaQL) [11] allows specification of a bundle query through linear constraints and objective functions over data attributes, but the users must know the exact query parameters: attribute names and their numerical bounds. This is particularly difficult for non-experts who must (i) learn the PaQL [11] syntax, (ii) know the database schema, (iii) determine the attributes of interest, (iv) have a good sense of the value distributions, and (v) translate their intent precisely to parameterized constraints and global objectives of a PaQL query. We show in Example 1 that even for experts, step (v) is particularly difficult for PaQL because constraints are over *bundle aggregates* (SUM, AVG, COUNT): users must reason at the bundle level—predicting what a SUM should be over a bundle of unspecified size—rather than at the tuple level as in SQL. For certain applications, such as extractive document summarization [85], an alternative is to specify the intent in natural language. However, as prior work shows, natural language is too generic to express specific user intents [20] and subsequent conversational tuning of intent is imprecise and frustrating for humans [87].

Example 1 (Synthesizing a package query). *Morpheus, a CS department Chair, is seeking to fill two tenure-track assistant professor positions to strengthen AI and Databases. Since he plans for the two new hires to jointly teach an "AI in Databases" course, he wants them to have reasonable collective teaching experience, but not too much, as that may indicate teaching-focused or senior candidates who are unlikely to accept a tenure-track entry-level position. He also wants to maximize the total recommendation score of the new hires based on their letters. This is a bundle retrieval problem: Morpheus must pick a set of two out of* 200 *candidates that best satisfy his criteria, as shown via the following under-specified package query:* [1]

```
Q1:  SELECT PACKAGE(*) FROM CANDIDATES
     SUCH THAT COUNT(*) = 2
     AND SUM(ai_score) is high
     AND SUM(db_score) is high
     AND SUM(teaching_score) is reasonable
     MAXIMIZE SUM(reco_score);
```

However, at this point, all Morpheus has are the raw application materials (resumes, statements, letters, etc.) of these candidates. Before querying the candidate database, he must first map each candidate's application materials to a 4-dimensional vector space: `ai_score`,

[1] While this toy example involves 2 hires over 4 criteria; faculty searches typically involve 5–10 candidates and 15+ criteria, yielding a combinatorial search space. E.g., choosing 5 from 200 candidates yields $\binom{200}{5} \approx 2.5$ billion possible bundles.

	AI ai_score	Databases db_score	Teaching teaching_score	Recommendation reco_score
Smith	0.8	0.4	0.1	0.4
Jones	0.4	0.7	0.2	0.9
Neo	0.4	0.5	0.4	0.8
Brown	0.5	0.4	0.4	0.9

Table 1: Partial list of candidates for Example 1.

db_score, teaching_score, *and* reco_score. *Morpheus decides to use an off-the-shelf embedding technique to obtain numeric scores for each of these dimensions for every candidate (Table 1). However, he now faces another problem: he does not know what constitutes an appropriate replacement for the underspecified values high and reasonable in* Q1. *Is* 0.7 *considered a high score for AI expertise? What value represents the reasonable range for teaching score? He can examine the value distributions of the embeddings, but still cannot determine which corresponding values would accurately capture his intended criteria.*

Example 1 highlights two key struggles in formulating package queries to express bundle query intent: (1) the lack of means to accurately map a database to a vector database with appropriate dimensions of interest, which primarily affects non-experts; and (2) the lack of knowledge to correctly parameterize the PaQL query, which affects experts and non-experts alike. In Example 1, Morpheus was interested in only 4 criteria; however, his struggle would be even worse if there were a larger number of criteria.

Our solution to Challenge 1: bundle-query by example. To overcome the usability challenge in the interface of bundle query intent specification, we build on the *Query by Example* (QbE) paradigm [90], which has seen significant success in traditional SQL querying [22] and other domains [20, 27, 29, 65] by lowering the barrier to task specification. We extend QbE to bundle queries (BQbE): the user provides a few exemplar data bundles to convey their query intents implicitly, thereby bypassing the construction of a precisely parameterized PaQL query. BQbE is particularly useful when users cannot articulate precise criteria but can instead provide representative examples of what they seek.

Example 2 (Parameterizing a package query). *Continuing from Example 1, Morpheus recalls that strong new hires resemble recent hires at top universities, such as {Trinity, Cypher} (University X) and {Link, Niobe, Seraph} (University Y). These exemplars excel in Databases and AI with reasonable collective teaching experience. Morpheus computes their scores across 4 dimensions using the same embedding of Example 1 (Table 2). Guided by the aggregated scores, he parameterizes the underspecified query* Q1 *to obtain:*

```
Q2:  SELECT PACKAGE(*) FROM CANDIDATES
     SUCH THAT COUNT(*) = 2
     AND SUM(ai_score) BETWEEN 0.8 AND 1.2
     AND SUM(db_score) BETWEEN 1.0 AND 1.3
     AND SUM(teaching_score) BETWEEN 0.7 AND 1.3
     MAXIMIZE SUM(reco_score);
```

Example 2 highlights a scenario where a user cannot directly specify precise criteria via PaQL query parameters but can provide valid *examples* to implicitly convey the criteria. These examples can be used to automatically learn user preferences and translate them to query parameters. However, another key challenge remains, which we highlight in Example 3.

	AI ai_score	Databases db_score	Teaching teaching_score	Recommendation reco_score
Example 1: University X hires				
Trinity	0.6	0.5	0.3	0.7
Cypher	0.2	0.8	0.4	0.9
SUM	**0.8**	**1.3**	**0.7**	**1.6**
Example 2: University Y hires				
Link	0.9	0.3	0.4	0.6
Niobe	0.2	0.3	0.6	0.8
Seraph	0.1	0.4	0.3	0.7
SUM	**1.2**	**1.0**	**1.3**	**2.1**
Morpheus' (implicit) intent				
	[0.8, 1.2]	**[1.0, 1.3]**	**[0.7, 1.3]**	**Maximize**

Table 2: Example bundles help discover query parameters.

bundle		ai	db	teaching	reco	valid?
b_1	{Smith, Jones}	1.2	1.1	0.3 (↓ 0.4)	1.3	**no**
b_2	{Smith, Neo}	1.2	0.9 (↓ 0.1)	0.5 (↓ 0.2)	1.2	**no**
b_3	{Smith, Brown}	1.3 (↑ 0.1)	0.8 (↓ 0.2)	0.5 (↓ 0.2)	1.3	**no**
b_4	{Jones, Neo}	0.8	1.2	0.6 (↓ 0.1)	1.7	**almost**
b_5	{Jones, Brown}	0.9	1.1	0.6 (↓ 0.1)	1.8	**almost**
b_6	{Neo, Brown}	0.9	0.9 (↓ 0.1)	0.8	1.7	**almost**

Table 3: No candidate bundle fully satisfies the constraints of Q2. For instance, b_6 violates the constraint "SUM(db_score) BETWEEN 1.0 AND 1.3", as its db_score (0.9) is 0.1 below the lower bound (1.0).

Example 3 (Ensuring query feasibility to obtain results). *Morpheus issues* Q2 *to a package query execution engine [11], but it returns no result. A close inspection reveals that indeed none of the 2-candidate bundles fully satisfies* Q2 *(Table 3). However, the last three bundles "almost" satisfy the constraints. If the* db_score *constraint had a slightly relaxed lower bound (0.9 instead of 1.0), then b_6 would be a valid bundle. Similarly, slightly relaxing the lower bound of the* teaching_score *constraint (0.6 instead of 0.7) would result in two additional valid bundles: b_4 and b_5. Then following the goal of maximizing the* reco_score*, the bundle b_5 would be the final result.*

Challenge 2: ensuring feasibility. While BQbE addresses the first challenge, it brings forth a second one that is particular to bundle retrieval and absent from traditional single-tuple QbE: constraints inferred from example bundles may yield queries that are *infeasible* over the target data, returning empty answers [50, 59], because aggregate constraints can be jointly unsatisfiable even when individually plausible, especially under distribution shift between the example and target domains. In Example 3, even with a fully formed PaQL query, with the desired constraints and specific parameters, it turned out to be *infeasible* over the target data (Morpheus' University). This happened because the data distributions differed between the example domains (Universities X and Y) and the target domain (Morpheus' University), a common scenario in practice [38, 68]. Existing package query execution systems [11] provide no further insight into how the query parameters interact with the target data, leaving users uncertain about how to tune the parameters to make the query feasible such that valid results are retrieved.

Our solution to Challenge 2: data-aware relaxation of query parameters. To ensure feasibility even when the original query is infeasible, we introduce *data-aware constraint relaxation techniques* that adapt the synthesized PaQL query to the target data

by adjusting constraint bounds. By analyzing the data distribution of the target database, we identify which constraints act as bottlenecks that prevent "almost-valid" bundles from being retrieved. This analysis guides us to relax the appropriate constraints, while minimizing deviation from the user's original intent.

Why existing approaches fall short. Three classes of relevant approaches exist, but all of them fail to support BQbE.

PaQL and QbE systems. PaQL engines [11] require fully parameterized queries and return empty results when infeasible; QbE systems [22, 90] reduce the parameter burden for single-tuple retrieval but do not extend to bundle queries with aggregate constraints.

Top-k retrieval. For BQbE, an alternative is to treat the entire example bundle as a single query vector and retrieve top-k similar tuples based on vector similarity [49, 72] to assemble the result bundle. However, this approach can miss globally optimal bundles and violate intended constraints, since top-k retrieval evaluates items independently using an implicit scoring function—without guarantees of global optimality—whereas bundle retrieval requires reasoning over combinations of tuples under globally enforced [89], multi-dimensional aggregate constraints. RAG systems inherit the same limitation, as they perform top-k nearest-neighbor search over individual tuples (but not their combinations) before passing results to a generative model. Another alternative is to match each tuple in the example bundle independently and combine their top matches to form the result bundle; yet this approach breaks down when the example and target bundle sizes differ. In summary, bundle retrieval is fundamentally an NP-hard combinatorial optimization problem, and similarity-based greedy top-k search fails to address it.

Conversational LLMs. An LLM prompted with examples (few-shot learning) could produce the bundle directly, but this requires reasoning over the entire target data and solving a combinatorial optimization problem, which LLMs handle unreliably [79]. An LLM could instead synthesize PaQL queries; however, the bounds would reflect an opaque mapping between the user's examples and constraint values. LLMs also have no native mechanism to detect infeasibility or relax bounds in an intent-preserving way, and per-query latency limits interactive use. In a pure NL interface, without examples to anchor the bounds, the user must rely on conversational refinement, which is imprecise and tedious [87].

Desiderata. Examples 1–3 and the limitations of alternative approaches motivate the desiderata of an ideal BQbE system:

- **D1** Ensure a user-friendly interface for specifying intents, where users can provide examples to implicitly convey their bundle query intents, without requiring any knowledge of the database schema and complex query syntax and parameters.
- **D2** Model the user intent implicit in the examples into a representation with explicit parameters, such as a PaQL query.
- **D3** Ensure feasibility of the synthesized PaQL query over the target database, yielding a non-empty result even when no bundle perfectly matches the user's intent.
- **D4** Provide an intuitive interface for iterative intent refinement, allowing users to adjust query constraints.

Ex2Bundle. We introduce Ex2Bundle, an example-driven data bundle retrieval system that (i) enables users to specify bundle query intent through example bundles, addressing D1; (ii) synthesizes a package query from these examples to transparently capture user intent, addressing D2; (iii) applies data-aware constraint relaxation to ensure feasibility of the synthesized query w.r.t. the target data domain, addressing D3; and (iv) provides an intuitive interface for interactive intent refinement, addressing D4.

Use cases. We now present three real-world applications where BQbE lowers the barrier for data bundle retrieval:

• *Supplier selection for business.* Businesses entering new markets struggle to identify a set of suppliers via explicit constraints over acceptable price ranges, inventory availability, or financial stability thresholds. Ex2Bundle can infer these constraints implicitly from examples of supplier bundles drawn from existing markets where the business is successful, adapt them to the target market, and retrieve an optimal set of suppliers that satisfies the constraints.

• *Focused snippet extraction.* In the context of text summarization, BQbE arises as *focused text snippet extraction*: given a "focus" in the form of several source document–summary pairs, where each summary is expressed as a set of extracted sentences (snippet), the goal is to select a snippet from a target document that is consistent with the examples. Ex2Bundle builds on our prior work SuDocu [20] to support personalized extractive summarization [86].

• *Playlist recommendation.* Existing playlist recommenders—e.g., Spotify's Discover Weekly [73], YouTube's Daily Discover [8], and Netflix's Top Picks [3]—rely on item-level or single-playlist similarity, which often fail to capture diverse tastes and cause user frustration [1]. Ex2Bundle allows users to provide example playlists reflecting their desired spread across genres, artists, tempo, etc., and generates playlists that satisfy their preferences.

Contributions. We make the following contributions:

- We formalize the problem of *example-driven bundle retrieval* and show how it translates to the package query framework (§2).
- We present Ex2Bundle, an end-to-end framework for *example-driven bundle retrieval* that synthesizes PaQL query constraints from user examples. We contribute a *data-aware constraint-bound relaxation technique* that ensures query feasibility. We further introduce design principles for an interactive slider-based interface for intent refinement, along with effective techniques to translate between sliders and constraint bounds (§3).
- We evaluate Ex2Bundle on two use cases—package queries over the TPC-H dataset [78] and focused text snippet extraction over real-world datasets [32, 86]—showing that Ex2Bundle achieves 100% constraint satisfaction while maintaining competitive objective scores, and outperforms retrieval and extractive baselines. Notably, Ex2Bundle remains competitive with LLM-based approaches without incurring any additional token or inference cost, while scaling to realistic workloads (§4).
- Our user study shows that Ex2Bundle achieves high user satisfaction, due to improved ease of use and customizable sliders, for the task of focused text snippet extraction (§5).

2 PROBLEM FORMULATION

In this section, we formalize the problem of example-driven bundle retrieval. We begin by introducing how we model the *source* and *target data*—from which bundles are retrieved—and *example*

Symbol	Description
$\mathbf{f} = \{f_1, \ldots, f_K\}$	Common schema over K attributes/features
$T_i^s \models \mathbf{f}$	A source data over the schema $\mathbf{f}$
$t \models \mathbf{f}$	A single tuple over the schema $\mathbf{f}$
$\mathbf{f}(t) = [f_1(t), \ldots, f_K(t)] \in \mathbb{R}^K$	Feature vector of tuple t
$E_i \subseteq T_i^s$	Example bundle for the source data T_i^s
$\mathbf{T}^s = \{T_1^s, \ldots, T_N^s\}$	Set of N source data
$\mathbf{E} = \{E_1, \ldots, E_N\}$	Set of N user-provided example bundles
$T^q \models \mathbf{f}$	A target data over $\mathbf{f}$ to retrieve a bundle from
$B \models \mathbf{f}$	A bundle over the schema $\mathbf{f}$
$\Theta = \{\langle \mathtt{lb}_j, \mathtt{ub}_j \rangle\}_{j=1}^K$	Constraint bounds for K attributes
$\mathbf{F}(B) = [F_1(B), \ldots, F_K(B)]$	Feature profile of bundle B
$\mathbf{F}(B) \vdash \Theta$	$\mathbf{F}(B)$ satisfies the constraints given by Θ
$\sigma(t) \mapsto \mathbb{R}$	Domain-specific tuple-level scoring function

Table 4: Table of notations. Bold letters denote sets or vectors.

bundles—which communicate the user's intent. We then show how bundle retrieval can be naturally modeled within the package query framework and define the problem of *example-driven package query synthesis* for bundle retrieval, our focus in this paper. Table 4 provides a summary of notations used throughout the paper.

Source and target data. In example-driven bundle retrieval, each example bundle is taken from a corresponding source data. In Example 2, {Trinity, Cypher} is an example bundle from the source data of University X and {Link, Niobe, Seraph} from the source data of University Y. One key requirement here is that the source data for all the example bundles must have the same schema, to allow for effective intent discovery across the shared attributes.

We define a schema $\mathbf{f}$ of a single table[2] over K numerical[3] attributes/features $\{f_1, \ldots, f_K\}$. Each source data T_i^s must conform to $\mathbf{f}$, denoted as $T_i^s \models \mathbf{f}$. In Table 2, the source data for all the Universities conform to the schema {`ai_score`, `db_score`, `teaching_score`, `reco_score`}. A tuple t over $\mathbf{f}$, denoted as $t \models \mathbf{f}$, is characterized by a feature vector $\mathbf{f}(t)$=$[f_1(t), \ldots, f_K(t)] \in \mathbb{R}^K$. In Table 2, Trinity's feature vector is $[0.6, 0.5, 0.3, 0.7]$. We use $T^q \models \mathbf{f}$ to denote the *target data* from which the user wants to extract their desired bundle. In Example 2, Table 1 represents the target data.

Example bundles. In example-driven bundle retrieval, users convey their intent via a set of *example bundles* $\mathbf{E} = \{E_1, \ldots, E_N\}$, where each $E_i \subseteq T_i^s$ consists of a subset of tuples from the corresponding source data T_i^s. In Example 2, $\mathbf{E}$ = {{Trinity, Cypher}, {Link, Niobe, Seraph}} and the source data set $\mathbf{T}^s = \{T^s_{\text{UofX}}, T^s_{\text{UofY}}\}$.

2.1 Example-driven bundle retrieval

We are now ready to (informally) define our problem:

Problem 2.1 (Example-Driven Bundle Retrieval). *Given a set of source data $\mathbf{T}^s$ over the same schema $\mathbf{f}$, corresponding example bundles $\mathbf{E}$, where $E_i \in \mathbf{E}$ is an example bundle for the source data $T_i^s \in \mathbf{T}^s$, and a single target data $T^q \models \mathbf{f}$, identify a bundle $B^* \subseteq T^q$ that (i) satisfies the "user intent" implicit in $\mathbf{E}$ w.r.t $\mathbf{T}^s$ and (ii) is the "best" among such bundles.*

However, this problem is underspecified: it is unclear how to model the user intent and define "best" or the notion of optimality.

[2]While our formalization focuses on a single-table schema, it is not an inherent limitation: we support relational databases by joining tables into a denormalized table, as shown in our experiments over the TPC-H dataset (§4).

[3]For unstructured data such as text, we derive numerical features through domain-specific methods such as topic modeling or learned vector embeddings (§3) to produce a structured representation under a fixed numerical schema.

2.1.1 Modeling user intent. Capturing user intent from example bundles may require complex models involving non-linear constraints. However, practical use cases share the following common structure: (i) the desired bundle is a *set* of items, (ii) quality of the bundle depends on some *aggregate* properties (e.g., total score) of the set, and (iii) the user preference is expressible as *bounds* on those aggregates. These align perfectly with *package query* [11]—which retrieves subsets of tuples from relational tables that collectively satisfy (linear) aggregate constraints while optimizing a (linear) global objective—as illustrated by `Q2` in Example 2. Thus, we model user intent as *linear constraints* on the bundle's *feature profile*, with an optional cardinality constraint on the bundle size. The benefit of this modeling is twofold: it provides *interpretability* —as linear constraints can be easily understood and inspected by humans [30, 56, 57, 63, 67, 69]—and *tractability*—it maps to Integer Linear Programming (ILP) with established solvers and foundations [11].

Feature profile of a bundle. To capture the feature-wise properties of a bundle B, we define its *feature profile* $\mathbf{F}$ as a vector obtained by aggregating, for each feature, the values of that feature across all tuples in B. Formally,

$$\mathbf{F}(B) = [F_1(B), \ldots, F_K(B)], \quad \text{where } F_j(B) = \mathcal{A}_{t \in B} F_j(t)$$

Here, $\mathcal{A}$ is an aggregate such as `SUM`. In Table 2, using `SUM` as the aggregate, the feature profile of the bundle {Trinity, Cypher} is $[0.8, 1.3, 0.7, 1.6]$—the column-wise sums for University X hires.

Formulating constraints. Following prior work in QbE [22], given a set of user-provided example bundles, we posit that the user's intended result bundle would exhibit a feature profile similar to that of the example bundles. Accordingly, we model the user's intent as a *set of bounded constraints over the feature profile* of the desired bundle, where the derivation of the bounds should be guided by the feature profiles of the example bundles in $\mathbf{E}$, the source data set $\mathbf{T}^s$, and the target data T^q. Formally:

$$\{\mathtt{lb}_1 \le \mathbf{F}_1(B) \le \mathtt{ub}_1, \ldots, \mathtt{lb}_K \le \mathbf{F}_K(B) \le \mathtt{ub}_K\}$$
$$\text{where } \mathtt{lb}_j \text{ and } \mathtt{ub}_j \text{ depend on } \mathbf{E}, \mathbf{T}^s, \text{ and } T^q \text{ for } 1 \le j \le K$$

We use $\Theta = \{\langle \mathtt{lb}_1, \mathtt{ub}_1 \rangle, \ldots, \langle \mathtt{lb}_K, \mathtt{ub}_K \rangle\}$ to denote constraint bounds for all K attributes over the schema $\mathbf{f}$. The notation $\mathbf{F}(B) \vdash \Theta$ denotes that the feature profile of the bundle B satisfies the constraint bounds specified by Θ, i.e., $\forall\, 1 \le j \le K,\ \mathtt{lb}_j \le F_j(B) \le \mathtt{ub}_j$.

2.1.2 Defining bundle optimality. The second issue in Problem 2.1 is that, when multiple candidate bundles satisfy Θ, a principled way is required to quantify their quality and ensure optimality. To this end, we assume knowledge of an application-specific tuple-level function $\sigma(t) \mapsto \mathbb{R}$, typically provided by a domain expert who configures the system for end users. We define the quality of a bundle B by aggregating σ over all $t \in B$, which naturally fits the linear objective function optimized by the PaQL framework. For example, this could correspond to maximizing the total recommendation score in Example 1, or minimizing the word count in text summarization. Alternatively, σ can be learned from user feedback on bundle quality, e.g., for playlist recommendation in music streaming.

2.2 Example-driven package query synthesis

With our models of user intent and bundle optimality, which naturally align with the package query framework, we reformulate

Problem 2.1 of example-driven bundle retrieval as that of synthesizing a package query—specifically, its parameters—from example bundles. The synthesized package query serves as a *mechanism* for efficiently retrieving the optimal result bundle.

Given a target data T^q, constraint bounds $\Theta = \{\langle \mathtt{lb}_j, \mathtt{ub}_j \rangle\}_{j=1}^{K}$, and (optional) cardinality constraint bounds $C = \langle \mathtt{lb}_c, \mathtt{ub}_c \rangle$, we fix the following parameterized package query:

$\mathrm{PQ}(T^q, \Theta, C)$: **SELECT** PACKAGE(*) **AS** B **FROM** T^q
SUCH THAT COUNT(B) **BETWEEN** $\mathtt{lb}_c$ **AND** $\mathtt{ub}_c$
AND $F_j(B)$ **BETWEEN** $\mathtt{lb}_j$ **AND** $\mathtt{ub}_j$ $\forall 1 \le j \le K$
MAXIMIZE $\underset{t \in B}{\mathcal{A}}\ \sigma(t)$

Here, $\mathcal{A}$ aggregates the tuples $t \in B$ using σ to compute the quality of B, which the package query aims to maximize as its objective. Without loss of generality, a minimization objective can be expressed as a maximization objective by simply negating the objective. The choice of aggregation functions (including those used to compute the feature profile $\mathbf{F}(B)$) and the scoring function σ is typically guided by the application domain. These are system parameters determined during setup and are not optimized over. We discuss how to choose the aggregation and scoring functions in §3.3. Also, without loss of generality, the cardinality constraint $\mathtt{lb}_c \le \mathtt{COUNT}(B) \le \mathtt{ub}_c$ can be subsumed into the feature-profile constraints by augmenting the feature profile with an additional dimension representing bundle cardinality. Hence, we do not explicitly include the cardinality constraint in the remainder of the paper.

In Example 2, Q2 is a package query over the schema $\mathbf{f}$={ai_score, db_score, teaching_score}, so $K = 3$. The aggregation used to compute the feature profile is SUM, and the objective is to maximize SUM(reco_score) (i.e., σ = reco_score).

While Morpheus manually derived the constraint bounds in Θ, the resulting query was infeasible over the target data T^q representing his university. Thus, the main requirement is to determine the parameter Θ, and use it to synthesize a package query that will retrieve the optimal bundle $B \subseteq T^q$, as specified in Problem 2.1.

Problem 2.2 (Example-driven package query synthesis). *Given a schema* $\mathbf{f}$*, a set of source data* $\mathbf{T}^s = \{T_1^s, \ldots, T_N^s\}$ *where each* $T_i^s \models \mathbf{f}$*, corresponding example bundles* $\mathbf{E}$*, a target data* $T^q \models \mathbf{f}$*, and a tuple-level scoring function* $\sigma : t \mapsto \mathbb{R}$*, together with the corresponding aggregator* $\mathcal{A}$*, determine a synthesis function* $G : (\mathbf{E}, \mathbf{T}^s, T^q) \mapsto \langle \mathbb{R}, \mathbb{R} \rangle^K$ *to derive the constraint bounds* Θ *such that:*

(1) ***Feasibility****: the package query* $PQ(T^q, \Theta)$ *returns a non-empty result over* T^q*, ensuring* $\exists\, B \subseteq T^q$ *s.t.* $\mathbf{F}(B) \vdash \Theta$.
(2) ***Optimality****: the retrieved bundle* $B^* = PQ(T^q, \Theta)(T^q)$ *maximizes the objective, i.e.,* $B^* = \underset{B \subseteq T^q \ s.t.\ \mathbf{F}(B) \vdash \Theta}{\arg\max}\ \underset{t \in B}{\mathcal{A}}\ \sigma(t)$.

The core challenge here is to determine the synthesis function $G : (\mathbf{E}, \mathbf{T}^s, T^q) \mapsto \langle \mathbb{R}, \mathbb{R} \rangle^K$ to derive the constraint bounds Θ by capturing the intent implicit in $\mathbf{E}$ while generalizing to the target data T^q. Unlike general query-by-example, we do not focus on synthesizing package queries with arbitrary structures, which would require inferring joins and aggregates. Instead, we focus on inferring the parameters of the constraint bounds Θ. This task is particularly challenging because the distribution of T^q may differ from $\mathbf{T}^s$: overly tight constraints (Example 2) can yield infeasible queries on the target data, while overly relaxed constraints allow bundles that deviate from user feasibility.

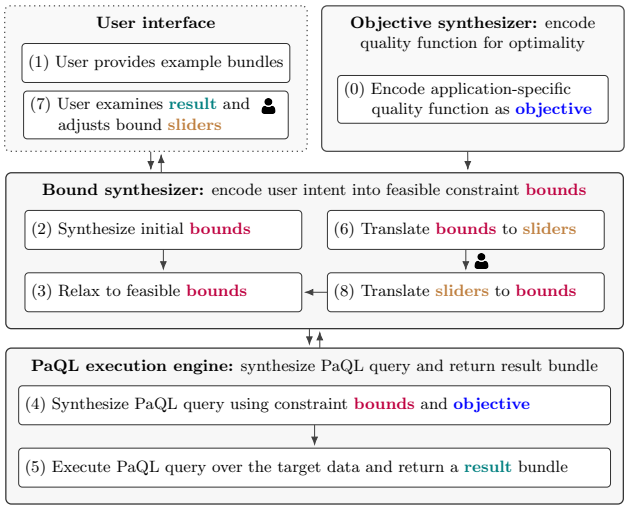


Figure 5: Ex2Bundle workflow: before end-user interaction, a domain expert provides a quality function to be used as the objective and specifies an aggregate function to be used in the constraints. (0) Ex2Bundle encodes the quality function as objective, (1) the user provides example bundles, from which Ex2Bundle formulates constraints using the expert-specified aggregate, (2) synthesizes initial constraint bounds and (3) relaxes them to ensure feasibility. A PaQL query is (4) formed using these bounds and the objective and (5) executed to retrieve a result bundle. For further intent refinement, (6) an interactive slider interface is generated, (7) the user inspects results and adjusts sliders, (8) Ex2Bundle maps adjustments back to bounds, and repeats (3)–(5).

3 THE EX2BUNDLE FRAMEWORK

Ex2Bundle consists of four main components (Fig. 5): (§3.1) *bound slider*, an interface that allows users to refine their intent by adjusting constraint bounds; (§3.2) *bound synthesizer*, which encodes user intent into *feasible* constraint bounds; (§3.3) *objective synthesizer*, which encodes an application-specific quality function as the objective of the PaQL query; and (§3.4) *PaQL execution engine*, which synthesizes and executes package queries to retrieve result bundles.

Running example for focused text snippet extraction. Throughout the rest of the paper, we use a running example centered on the task of *Focused Text Snippet Extraction* (FTSE), a primary application through which we evaluate Ex2Bundle (§4). Because Ex2Bundle requires numeric features and text is non-numeric, we apply *contextualized topic modeling* (CTM) [6] with sentence embeddings from SBERT [76] to map sentences into a numeric vector space of topic-based features. Under the resulting topic model defined over schema $\mathbf{f}$, $f_j(t) \in [0, 1]$ denotes the relevance of a sentence t to topic f_j, and $\mathbf{f}(t)$ denotes the vector representation of t in the topic space $\mathbf{f}$. For ease of understanding, we use the following, more specific and domain-relevant terms for FTSE, instead of generic terms:

General term	FTSE term	General term	FTSE term
source/target data	source/target document	tuple	sentence
bundle	snippet or summary	schema	topic model
attribute/feature	topic	feature profile	topic profile

Example 4. *Consider a topic model* $\mathbf{f}$ *over* Demographics, Geography, *and* Economy *applied to documents describing U.S. states. Let* T_{UT}^s *denote the Utah document and* $E_{UT} = \{t_{UT}^1, t_{UT}^2, t_{UT}^3\} \subseteq T_{UT}^s$ *be three*

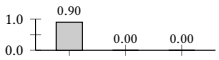
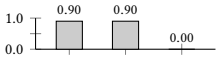
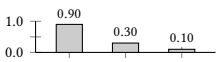
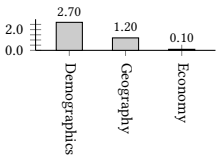
Sentence/snippet	Topic score/profile
(t^1_{UT}) The largest ancestry groups in the state are: 26.0% English, 11.9% German, [...]	
(t^2_{UT}) In comparison to all the U.S. states and territories, Utah, with a population of just over three million, is the 13th-largest by area, the 30th-most populous, and the 11th-least densely populated.	
(t^3_{UT}) St. George was the fastest-growing metropolitan area in the United States from 2000 to 2005.	
(E_{UT}) The largest ancestry groups in the state are: 26.0% English, 11.9% German, [...]. In comparison to all the U.S. states and territories, Utah, with a population of just over three million, is the 13th-largest by area, the 30th-most populous, and the 11th-least densely populated. St. George was the fastest-growing metropolitan area in the United States from 2000 to 2005.	

Table 6: Topic scores for sentences t^1_{UT}, t^2_{UT}, and t^3_{UT}; and topic profile for the example snippet $E_{\text{UT}} = \{t^1_{\text{UT}}, t^2_{\text{UT}}, t^3_{\text{UT}}\}$.

sentences (Table 6). Under $\mathbf{f}$, t^1_{UT} *maps to* $\mathbf{f}(t^1_{UT}) = [0.90, 0.00, 0.00]$, *indicating a demographics focus;* t^2_{UT} *to* $[0.90, 0.90, 0.00]$, *covering demographics and geography; and* t^3_{UT} *to* $[0.90, 0.30, 0.10]$, *primarily demographics with weaker geography and economy associations.*

3.1 Bound slider interface for intent refinement

A key requirement (D4 in §1) for Ex2Bundle is an intuitive user interface for iterative intent refinement. A simple approach is to let users directly edit the synthesized constraint bounds (Ex2Bundle's bound synthesis is discussed in §3.2). However, non-experts often do not know the precise mapping between their intent and constraint bounds (as in Example 1). Based on studies [15, 23, 40, 70, 74, 83] establishing that humans are better at relative judgments (more vs. less) than absolute ones (assigning ratings) [77], we introduce a single-parameter slider that abstracts the raw bounds.

Fig. 7 shows the slider interface, where each position corresponds to a pair of upper and lower bounds, and the neutral point corresponds to the feasible bounds synthesized from user examples (details in §3.2). The slider instance in Fig. 7 depicts the topic Demographics, where the neutral point 0 corresponds to the feasible bounds $\langle 1.50, 2.70\rangle$. The slider further encodes other representations on a scale from −100 to +100 around the neutral point to allow fine-grained adjustments. Upon reviewing the result bundle, users simply indicate whether they want more (less) relevance to the topic by moving the slider right (left). This design also reduces the number of controllable parameters, consistent with HCI findings that simpler control spaces are cognitively less demanding [35].

Mapping between slider positions and bounds. We use γ_j to denote the position of the slider for topic f_j. We map the initial feasible bounds $\langle \texttt{lb}_j, \texttt{ub}_j\rangle$—synthesized from user examples—to the midpoint of the slider at γ_j=0. For a target document T^q, the minimum and maximum attainable bundle scores for topic f_j are 0 and $F_j(T^q)$, respectively. Let us use $\texttt{mx}$ to denote $F_j(T^q)$ for simplicity. These two extreme values—0 and $\texttt{mx}$—map to the lower bound of the left-most slider position and upper bound of the right-most slider position. A key challenge here is that both sides of the midpoint of the slider have an equal number of positions (100 on each side); however, the distance between the initial feasible bounds and the two extreme bounds is not necessarily equal. Thus, for slider positions

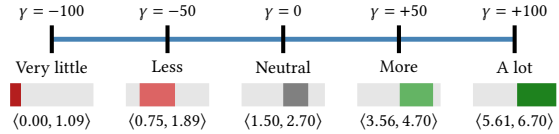


Figure 7: Slider for Demographics. Users can adjust topic emphasis from −100 (very little) to +100 (a lot). Each slider position maps to specific constraint bounds; the neutral position (0) corresponds to the feasible bounds synthesized (and relaxed) from user examples.

left of the midpoint, we determine the *lower* bound by interpolating based on the position's relative distance between the left-most endpoint (γ_j=−100) and the midpoint (γ_j=0). Symmetrically, for positions right of the midpoint, we determine the *upper* bound by interpolating based on the position's relative distance between the right-most endpoint (γ_j=+100) and the midpoint (γ_j=0). Assuming a constant width $\texttt{w}$ for all bounds, the mappings are as follows:

Slider Position	Lower bound	Upper bound	γ_j
(left-most)	0	$\texttt{w}$	−100
(left of the center)	$\texttt{lb}_j \cdot \frac{(100-x)}{100}$	$\texttt{lb}_j \cdot \frac{(100-x)}{100} + \texttt{w}$	$-x$
(center)	$\texttt{lb}_j$	$\texttt{ub}_j$	0
(right of the center)	$\texttt{ub}_j + \frac{(\texttt{mx}-\texttt{ub}_j)\cdot x}{100} - \texttt{w}$	$\texttt{ub}_j + \frac{(\texttt{mx}-\texttt{ub}_j)\cdot x}{100}$	$+x$
(right-most)	$\texttt{mx} - \texttt{w}$	$\texttt{mx}$	+100

A question still remains: how to determine the value of $\texttt{w}$? Should it be constant for all slider positions? We use the width of the neutral position $(\texttt{ub}_j - \texttt{lb}_j)$ as a guide to derive the bound width of slider position at $\gamma_j = x$ with the formula: $\texttt{w}(x) = e^{-\frac{\alpha \cdot |x|}{100}} \cdot (\texttt{ub}_j - \texttt{lb}_j)$.

Here, α is a tunable parameter between 0 and ∞, which determines how aggressively bound widths are narrowed as the slider moves farther away from the neutral position. When α=0, all slider positions use the same width as the neutral position, i.e., $(\texttt{ub}_j - \texttt{lb}_j)$. As α increases, the width shrinks more rapidly as the slider moves away from the neutral position, resulting in increasingly tighter bounds for extreme values of γ_j. Regardless, Ex2Bundle automatically "snaps" the slider to a feasible position in case the user's adjustment leaves it at an infeasible state. Note that the bounds represented by different slider positions can be overlapping (Fig. 7).

Overall, the slider interface offers the following benefits: (1) *simplicity* by reducing the number of controls, (2) *semantic clarity* through an intuitive scale from less (negative) to more (positive), (3) *feasibility preservation* by automatically snapping user adjustments to a feasible state, and (4) *reversibility* by allowing users to reset to the original (or any intermediate) intent. Users can still adjust the bounds directly if they wish, but the slider provides a more intuitive alternative that bypasses the complex underlying numeric bounds. We evaluate the effectiveness of the slider interface in §5.

3.2 Encoding intent into feasible constraints

The bound synthesizer (i) synthesizes initial, potentially infeasible, bounds from user examples to form constraints, (ii) relaxes the initial bounds to ensure feasibility, (iii) exposes the feasible bounds via the slider interface for iterative refinement, and (iv) translates the adjusted sliders back to feasible bounds after refinement.

3.2.1 Synthesizing the initial bounds. We extend SQuID [22], which learns user intent from example tuples, to a bundle-level granularity. Given example bundles $\mathbf{E} = \{E_i\}_{i=1}^{N}$, we use SUM as the aggregate to

Algorithm 1: Ex2Bundle bound relaxation algorithm

Input : Initial bounds Θ_{init}, schema $\mathbf{f}$, target data T^q
Relaxation parameters μ and τ
Output : Feasible bounds Θ to construct the PaQL constraints

1 $\Theta \leftarrow \Theta_{\text{init}}$
2 #attempts $\leftarrow 0$ // Initialize #attempts
3 **while** $PQ(\Theta, T^q) = \emptyset$ // While query is infeasible
4 **do**
5 $\Theta_{\text{violated}} = \text{IdentifyViolations}(\Theta, T^q)$ // Violated constraints
6 $\Theta_{\text{relaxed}} = \emptyset$ // Relaxed constraints
7 $\rho \leftarrow \exp\left(\mu \cdot \max\left(1, \left\lfloor \frac{\#\text{attempts++}}{\tau} \right\rfloor\right)\right)$ // Relaxation step multiplier
8 **foreach** $\langle \text{lb}_j, \text{ub}_j \rangle \in \Theta_{violated}$ **do**
9 $\epsilon_j \leftarrow \frac{F_j(T^q)}{|T^q|}$ // Relaxation step size
10 $\text{lb}_j \leftarrow \max(\text{lb}_j - \rho \cdot \epsilon_j, \ 0)$ // Decrease lower bound
11 $\text{ub}_j \leftarrow \min(\text{ub}_j + \rho \cdot \epsilon_j, \ F_j(T^q))$ // Increase upper bound
12 $\Theta_{\text{relaxed}} = \Theta_{\text{relaxed}} \cup \{\langle \text{lb}_j, \text{ub}_j \rangle\}$
13 $\Theta = (\Theta \setminus \Theta_{\text{violated}}) \cup \Theta_{\text{relaxed}}$ // Update constraint bounds
14 **return** Θ

compute their feature profiles $\{\mathbf{F}(E_i)\}_{i=1}^N$. The choice of aggregate is application-specific and our framework can support any linear aggregate such as `COUNT`, `MIN`, `MAX`, etc. However, `SUM` is often preferred since `AVG` cannot distinguish single- from multi-tuple bundles, and `MAX/MIN` are outlier-sensitive. We derive the initial constraint bounds $\Theta_{\text{init}} = \{\langle \text{lb}_j, \text{ub}_j \rangle\}_{j=1}^K$ by taking the topic-wise minimum and maximum of the feature profiles of bundles in $\mathbf{E}$:

$$\text{lb}_j = \min_{1 \le i \le N} F_j(E_i), \quad \text{ub}_j = \max_{1 \le i \le N} F_j(E_i)$$

Example 5. *Consider 3 example snippets E_{UT}, E_{AZ}, and E_{WA} over the source documents T^s_{UT}, T^s_{AZ}, and T^s_{WA} that represent Wikipedia pages for Utah, Arizona, and Washington, respectively. The topic profile of E_{UT}, $\mathbf{F}(E_{UT}) = \mathbf{f}(t^1_{UT}) + \mathbf{f}(t^2_{UT}) + \mathbf{f}(t^3_{UT}) = [2.70, 1.20, 0.10]$, as shown in the last row of Table 6. The topic profiles for the example snippets, along with the initial constraint bounds are given below, which results in $\Theta_{init} = \{\langle 1.50, 2.70 \rangle, \langle 0.90, 1.20 \rangle, \langle 0.10, 0.40 \rangle\}$.*

	F_1 (Demographics)	F_2 (Geography)	F_3 (Economy)
$\mathbf{F}(E_{UT})$	2.70 ↑	1.20 ↑	0.10 ↓
$\mathbf{F}(E_{AZ})$	1.80	0.90 ↓	0.40 ↑
$\mathbf{F}(E_{WA})$	1.50 ↓	1.00	0.30
lb *(min)*	1.50	0.90	0.10
ub *(max)*	2.70	1.20	0.40

Remark on constraint expressivity. Ex2Bundle supports any tuple-level constraint that standard SQL supports, such as equality predicates involving categorical attributes, logical conditions, and constraints over derived (potentially non-linear) attributes. Techniques for deriving such tuple-level constraints are addressed in prior work [22]. Ex2Bundle also supports more complex bundle-level constraints, such as interactions among aggregated attributes, and special cases of bounded constraints, such as equalities (by setting $\text{lb} = \text{ub}$) and one-sided inequalities (by setting $\text{lb} = -\infty$ or $\text{ub} = \infty$). Ex2Bundle supports any linear aggregate within the bundle-level constraints. The choice of the aggregate is application-specific and should be specified by a domain expert. In the absence of domain expertise, an LLM can suggest a suitable aggregate based on the task context. We note that practical applications may involve non-linear constraints, such as Max-Min diversity [4]. However, this is a limitation of PaQL, which Ex2Bundle relies on, not of Ex2Bundle itself. In fact, the underlying ILP solvers such as IBM CPLEX [37] support certain classes of quadratic constraints with specific properties (e.g., convexity and semi-definiteness) and Ex2Bundle can potentially support such constraints once PaQL provides support for them.

3.2.2 Bound relaxation to ensure feasibility. Overly tight bounds can yield infeasible queries that return no valid bundle (Example 3), particularly when bounds are misaligned with the target data's feature distribution under distributional shift (Example 3). Infeasibility arises from two sources. The first is *insufficient feature coverage*: the target data cannot satisfy a feature's lower bound even when all tuples are selected. For example, if the source data (e.g., Utah, Arizona, & Washington) and example snippets emphasize `national parks` (e.g., Bryce, Grand Canyon, Olympics), but the target document is Kansas, which contains no national parks, its maximum contribution to that topic is 0, making any positive lower bound (e.g., 0.10) infeasible. The second is *atomicity*: tuples are indivisible, so selecting any single tuple may violate an upper bound. For instance, if every target sentence has a topic score of at least 0.15, an upper bound of 0.10 is infeasible because no tuple can be partially selected.

Bound relaxation algorithm. We address infeasibility via *bound relaxation*, which iteratively widens constraint bounds until the resulting package query becomes feasible. Algorithm 1 outlines this process, which repeats until feasibility is achieved (lines 5–13). If the initial constraint bounds Θ_{init} are infeasible (line 3), we first identify the violated constraints Θ_{violated} (line 5) using ILP solver signals (e.g., IBM CPLEX [37]). To preserve the original intent, we relax only the violated constraints. Specifically, we decrease lb_j toward 0 (line 10) and increase ub_j toward $F_j(T^q)$ (line 11). We determine the symmetric relaxation amount at each iteration using $\rho \cdot \epsilon_j$ (lines 10 & 11), where the step multiplier ρ (line 7) controls the relaxation rate based on the number of prior attempts, and the step size ϵ_j (line 9) is a target-data-specific parameter. We also considered two alternative relaxation techniques, including directional relaxation, which leverages the ILP solver's information about whether the lower or upper bound causes a violation. However, as shown in §4.6.2, these alternatives never improved bundle quality over symmetric relaxation and had comparable runtime for typical relaxation severity. We therefore adopt symmetric relaxation due to its simplicity and empirical effectiveness.

Relaxation step multiplier, ρ. It controls the relaxation rate and is defined as $e^{\mu \cdot \max(1, \lfloor \#\text{attempts}/\tau \rfloor)}$. Here, μ (default 0.5) sets the base relaxation rate, while τ (default 10) controls how frequently ρ is boosted. Consequently, ρ increases exponentially every τ attempts, enabling faster escape from infeasible regions.

Relaxation step size, ϵ_j. While ρ determines the rate of relaxation, different features exhibit varying score distributions within T^q. To account for this, we derive a feature-specific step size from the target data distribution and set $\epsilon_j = \frac{F_j(T^q)}{|T^q|}$, which corresponds to the average contribution of feature f_j per tuple in T^q. Since the output bundle is a subset of the target data T^q, the appropriate relaxation magnitude depends on the empirical scale of each feature in T^q. We therefore derive ϵ_j from T^q: a fixed step can be too small or too large for features with different value ranges. Computing ϵ_j per feature automatically adapts the step size to the scale of that feature, even when feature values are not normalized.

Example 6. *Consider the California document T^q_{CA} over 5 sentences ($|T^q_{CA}| = 5$), where $F_2(T^q_{CA}) = 0.60$. The initially synthesized constraint is $0.90 \leq F_2(B) \leq 1.20$ (Example 5). This is infeasible since the maximum achievable score over T^q_{CA} is 0.60, requiring relaxation. For the first 10 iterations, $\rho = e^{0.5\times1} \approx 1.65$ and $\epsilon = \frac{0.60}{5} = 0.12$. Thus, the first iteration relaxes the bounds to $\max(0.90-1.65\times0.12, 0) = 0.70 \leq F_2(B) \leq min(1.20+1.65\times0.12, 0.60) = 0.60$. This is still infeasible, so the second iteration yields $\max(0.70-1.65\times0.12, 0) = 0.50 \leq F_2(B) \leq \min(0.60+1.65\times0.12, 0.60) = 0.60$, at which point the constraint becomes feasible and relaxation stops. The upper bound is not relaxed due to it already being at max.*

3.2.3 Bound relaxation after slider interaction. Since slider adjustments may re-introduce infeasibility, we re-relax the bounds afterwards using Algorithm 1. Once feasible bounds are obtained, we snap the slider to the position whose bounds are closest to the feasible bounds and subsume them, ensuring transparency to the user. Unlike the initial relaxation, we have an advantage here since we only need to relax bounds for a single constraint, with all other constraint bounds fixed. Therefore, if previously feasible bounds $\langle \mathtt{lb}^*_j, \mathtt{ub}^*_j \rangle$ are known, we use them as an additional reference during relaxation. Specifically, we reduce $\mathtt{lb}_j$ until $\max(0, \mathtt{lb}^*_j)$ (line 10 of Algorithm 1) and increase $\mathtt{ub}_j$ until $\min(\mathtt{ub}^*_j, F_j(T^q))$ (line 11). Additionally, if the user requests termination of relaxation before the process finishes, we move the slider to the closest feasible bounds.

3.3 Devising a quality function for optimality

As discussed in §2.1.2, the quality function defines the notion of optimality used to break ties among multiple valid bundles. This becomes particularly important as bounds relaxation often yields multiple valid bundles. The quality function is application-specific and is typically specified by a domain expert prior to user interaction with an instance of Ex2Bundle, as shown in step (0) in Fig. 5. Given (i) a tuple-level scoring function $\sigma : t \mapsto \mathbb{R}$, which assigns a numeric score to a tuple t, and (ii) an application-specific aggregate function $\mathcal{A}$, the quality of a candidate bundle B, $quality(B) = \mathcal{A}_{t\in B}\, \sigma(t)$. Below, we present example quality functions for the three applications discussed in §1.

Supplier selection for business. Cost and *reliability* are common quality functions when selecting a bundle of suppliers for business expansion into a new country. For cost, a natural aggregate is `SUM`, with the objective of minimizing total cost, i.e., $\Sigma_{t\in B}\mathtt{cost}(t)$. For reliability, suitable aggregates include `AVG` or `MIN`, depending on the preference: maximizing average reliability, or maximizing the minimum reliability among the selected suppliers.

Focused snippet extraction. Informativeness and *conciseness* are common quality functions when extracting snippets from a text document. Sentence-level informativeness can be modeled using term frequency within a sentence t, weighted by inverse document frequency in the corpus (TF-IDF), which assigns higher scores to sentences containing rarer words w.r.t the corpus. Informativeness can also be approximated using structural signals such as the presence of numbers, citations, or hyperlinks, which often indicate factual or reference-rich content. Conciseness can be modeled using the word or character count of a sentence, with the objective of minimizing the total number of words or characters in a snippet. In both cases, the most appropriate aggregate is `SUM`. Alternatively, conciseness can be modeled by minimizing the `COUNT` of sentences in a snippet.

Playlist recommendation. Popularity and *diversity* are common quality functions in playlist recommendation. Popularity can be modeled using the number of weekly plays or chart rankings of a song, with the objective of maximizing overall popularity using aggregate `AVG`, i.e., $\mathtt{AVG}_{t\in B}\ \mathtt{popularity}(t)$. Diversity can be modeled using pairwise dissimilarity between songs based on features such as genre, mood, or artist. This can be captured via a distance function over song features, with the objective of maximizing diversity within the playlist. A suitable aggregate is `MIN` over all pairwise distances, i.e., $\mathtt{MIN}_{t_i,t_j\in B, i\neq j}\mathtt{distance}(t_i, t_j)$, and maximizing this ensures that even the most similar pair of songs in the playlist remains sufficiently dissimilar. While in this paper we assume a linear, tuple-wise scoring function, Ex2Bundle can support quadratic objectives via CPLEX's quadratic programming or MILP linearization.

3.4 Package query execution

For the package query execution engine, we use PaQL [11], a query engine for declarative package queries. PaQL translates queries into Integer Linear Programs (ILPs), which are then solved using off-the-shelf solvers such as IBM CPLEX [37]. For large target data with many tuples, solving the ILP becomes computationally expensive due to the combinatorial nature of the search space. To address this, Ex2Bundle first PreFilters T^q to at most $10x$ tuples most similar to the examples, where x is the average example size; the ILP then enforces the constraints and selects the optimal bundle from that pool. Since this reduction shrinks as x grows, Ex2Bundle also applies PaQL's SketchRefine [11], which solves the ILP over cluster representatives with a guaranteed $(1+\epsilon)^6$ approximation of the optimal bundle. Empirical results are in §4.5.

Ex2Bundle formulates a PaQL query that encodes the constraint bounds Θ (§3.2) as constraints over packages (analogous to snippets in FTSE or bundles in general) and defines the optimization objective using the scoring function σ together with the aggregate $\mathcal{A}$ (§3.3), as shown in §2.2. It then executes the package query, and, if feasible, returns the optimal bundle. The PaQL engine also informs the bound synthesizer (§3.2) about any violated constraints when the query is infeasible in line 5 of Algorithm 1.

Example 7. *For the constraint bounds $\Theta = \{\langle 1.50, 2.70\rangle, \langle 0.50, 0.60\rangle, \langle 0.10, 0.40\rangle\}$ (derived in Example 5 and relaxed in Example 6) and the target document T^q_{CA}, we want the snippet with the minimum total word count. The function $\mathtt{word\_count} : t \mapsto \mathbb{N}^+$ returns the number of words in a sentence t. The synthesized PaQL query is as follows:*

$\mathrm{PQ}(T^q_{CA}, \Theta)$:
```
SELECT PACKAGE(*) AS B FROM T^q_CA
SUCH THAT F_1(B) BETWEEN 1.50 AND 2.70
      AND F_2(B) BETWEEN 0.50 AND 0.60
      AND F_3(B) BETWEEN 0.10 AND 0.40
MINIMIZE SUM_{t∈B} word_count(t)
```

Remark: Example-based inference assumes the provided examples are correct, yet in practice they may be noisy or insufficiently representative. Ex2Bundle mitigates this by letting users adjust the intent sliders directly. Alternatively, examples that differ substantially from the others could be flagged for the user to verify, as in prior work on disambiguating user intent in programming by example [52]; we leave this to future work.

4 EXPERIMENTAL RESULTS

We now present experimental results addressing two sets of research questions. RQ1–RQ3 evaluate to what extent Ex2Bundle's PaQL synthesis algorithm meets the three *correctness criteria* for BQbE: constraint satisfaction, alignment with user intent, and consistency between constraints and bundles (detailed in §4.1). RQ4–RQ6 focus on scalability, practical efficiency, and robustness aspects.

RQ1 To what extent do the bundles returned by Ex2Bundle satisfy the synthesized constraints? How does it compare to alternatives? (§4.2).
RQ2 To what extent do the bundles returned by Ex2Bundle align with user intent? How does it compare to LLM-based baselines? (§4.3.1).
RQ3 Does moving farther from the synthesized constraints produce increasingly less aligned bundles with the intended bundle? (§4.4).
RQ4 How does Ex2Bundle's runtime scale with the size of example bundles and the size and dimensionality of the documents? (§4.5).
RQ5 How often is constraint relaxation triggered as example size increases, and how effective is it for BQbE? (§4.6.1).
RQ6 How robust is Ex2Bundle to varying degrees of skewness in the target document's topic distribution? (§4.7).

Implementation. Ex2Bundle uses a Python backend—with Gensim for frequency-based topic modeling, SBERT [76] for semantics-aware topic modeling, IBM CPLEX 20.1 via docplex for ILP solving, and a JavaScript/Bootstrap frontend for the slider interface. We ran experiments on a shared compute cluster, utilizing 4 Intel Emerald Rapids CPU cores, 64 GB RAM, and an NVIDIA H200 GPU.

Datasets. We evaluate Ex2Bundle in two applications: (1) supplier selection over TPC-H [78], using a synthesized view of `Supplier` (100 rows), `Partsupp` (8,000 rows), and `Nation` (25 rows) at scale factor 0.01, with five attributes: `price`, `availability`, `balance`, `region_europe`, and `region_america`; and (2) focused text snippet extraction (FTSE) over two datasets: (a) SubSumE [86], containing 275 (user, intent) pairs, each with 8 manually curated summaries; we use 5 summaries as examples and the remaining 3 for test; and (b) CNN/DailyMail [33], containing human-written highlights spanning four news categories. Since these highlights are abstractive, we use ChatGPT-4o to align each with source-article sentences, capping the snippet at 10 sentences or 30% of the article's sentences.

Baselines. We consider the following baselines, where k is the average example size:

- *Greedy* greedily selects k tuples by decreasing objective score.
- *Cluster* uses K-means clustering over the attribute space, and selects the tuple with highest objective score from each cluster.
- *Random* selects k tuples uniformly at random.
- *Top-k* selects the k most similar sentences to the examples by SBERT [76] cosine similarity.
- *SuDocu* [20] is our prior FTSE system, using Latent Dirichlet Allocation [7] topic distributions.
- *BERTSumExt* [48] is an extractive summarizer adapted for BQbE by pre-filtering sentences based on similarity to the examples.
- *MemSum* [25] is a reinforcement-learning extractive summarizer for long documents, adapted via the same pre-filtering.
- *ChatGPT-4o* [61] is prompted for FTSE.
- *LLM-agent* uses Llama-3.3-70B-Instruct-FP8 [53] for agent-based PaQL synthesis (including bounds), relaxation, and execution.
- *LLM-Ex2Bundle* uses the same LLM for PaQL synthesis, but with Ex2Bundle's constraint relaxation and execution.

More experimental details and results are in our technical report [14]

4.1 Correctness Criteria & Metrics

We define the correctness criteria for BQbE along three axes:

Constraint satisfaction provides a formal correctness guarantee for feasible queries by ensuring that the returned bundle satisfies the synthesized constraints. To measure the degree of constraint satisfaction, we use *Constraint Satisfaction Rate (CSR)*, the fraction of synthesized constraints satisfied by the retrieved bundle.

Intent alignment is a key correctness criterion that ensures the result bundle produced by a BQbE system closely matches the ground truth in terms of user intent. We use application-specific metrics to measure intent alignment: (1) *Objective score*: composite of TPC-H attributes: sum of `price`, `availability`, and `balance`; (2) *ROUGE-1/2/L* [44]: F_1 scores measuring unigram, bigram, and longest-common-subsequence overlap between result and ground truth; and (3) *Semantic Similarity (SS)*: cosine similarity between average SBERT embeddings of result and ground-truth snippets.

Constraint-bundle alignment consistency evaluates whether deviations from the synthesized PaQL Q produce corresponding deviations in the resulting bundle. Specifically, as a query Q' moves farther from Q, its resulting bundle should become increasingly less aligned with the intended bundle. We measure this using Pearson's correlation coefficient (PCC) between the Hausdorff distance [66] between the feasible regions induced by two queries and the cosine distance between the SBERT embeddings of the resulting bundles.

4.2 Constraint satisfaction effectiveness

We evaluate RQ1 (constraint satisfaction) for the task of supplier selection on TPC-H. We construct 3 example supplier bundles representing diverse procurement strategies (conservative, price-focused, and balanced); Ex2Bundle infers constraints from these examples per §3.2, synthesizes a PaQL query, and executes it to retrieve a result bundle. We compare Ex2Bundle against 3 baselines: constraint-agnostic *Greedy* (top-k highest-scoring tuples), *Cluster* (best tuples from each of the k clusters), and *Random* (k random tuples).

Table 8 shows that Ex2Bundle satisfies all constraints (CSR = 100%) with a competitive average objective score (11.72). Greedy achieves the highest objective score (15.61) but satisfies on average only 33.3% of the inferred constraints, indicating that objective-only optimization fails to enforce constraints. Cluster outperforms greedy (CSR = 66.7%); however, it fails to satisfy many constraints. Random has a higher CSR (70.0%) than Greedy and Cluster, but with a much lower objective score (10.48), highlighting the importance of jointly optimizing both constraint satisfaction and objective.

- Greedy and Cluster have lower constraint satisfaction rate (CSR) while Random has low objective score.
- Ex2Bundle achieves 100% CSR with a competitive objective score as it accounts for both objective and constraints.

System	CSR (%) ↑	Average Objective Score ↑	Runtime (s) ↓
Random	<u>70.0</u>	10.48	**0.0005**
Greedy	33.3	**15.61**	<u>0.0024</u>
Cluster	66.7	<u>14.04</u>	0.4712
Ex2Bundle	**100.0**	11.72	0.0207

Table 8: Ex2Bundle achieves full constraint satisfaction with a competitive objective score. Bold = best; <u>underlined</u> = second-best.

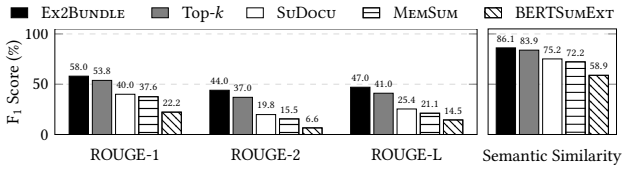


Figure 9: For focused text snippet extraction on the SubSumE dataset, Ex2Bundle outperforms all other baselines across all metrics.

Intent	System	Semantic Similarity ↑
Intent 1: What about this state's arts and culture attracts you the most?	ChatGPT-4o	0.64
	Ex2Bundle	**0.88**
Intent 2: What are some of the most interesting things about this state?	ChatGPT-4o	0.65
	Ex2Bundle	**0.76**

Table 10: Ex2Bundle outperforms ChatGPT-4o on both SubSumE intents; the advantage narrows on the more general one.

Method	R1	R2	RL	SS	Time (s)	#Relaxation	#Tokens
LLM-agent	0.35	0.17	0.23	0.68	923.60	2.11	104,127
LLM-Ex2Bundle	0.39	0.22	0.27	0.74	29.11	2.67	6,536
Ex2Bundle	**0.56**	**0.43**	**0.48**	**0.85**	**0.84**	**1.67**	**0**

Table 11: Ex2Bundle outperforms LLM-agent and LLM-Ex2Bundle.

4.3 Alignment with user intent

4.3.1 Comparison with non-LLM baselines. For RQ2, we use the SubSumE dataset and evaluate Ex2Bundle against Top-k, SuDocu, BERTSumExt, and MemSum. We report ROUGE-1/2/L F_1 scores and SBERT-based semantic similarity (SS) between retrieved and ground-truth snippets, together with per-query runtime.

Fig. 9 shows that Ex2Bundle consistently outperforms all baselines on ROUGE and SS. SuDocu is a BQbE system and outperforms non-BQbE extractive summarizers MemSum and BERTSumExt, but its topic modeling is inferior to Ex2Bundle and Top-k, both of which use semantics-aware topic modeling.

- Ex2Bundle outperforms all baselines in terms of ROUGE & SS.
- Systems that use semantics-aware topic modeling (Ex2Bundle & Top-k) outperform SuDocu and non-BQbE extractive summarization (BERTSumExt & MemSum).

4.3.2 Comparison with LLM-based baselines as direct snippet extractor. We compare Ex2Bundle against LLM-based baselines prompted to directly retrieve snippets from a target document given a few example snippets. We use ChatGPT-4o as a RAG baseline that performs few-shot learning from the examples. We evaluate two intents from SubSumE—one more specific and one less specific—to assess how each approach handles varying intent specificity (Table 10). Ex2Bundle outperforms ChatGPT-4o on both intents, but the advantage narrows for the less specific intent. For generic intents over CNN/DailyMail, ChatGPT-4o outperforms Ex2Bundle, which highlights that generic intent is too broad for example-driven retrieval, making Ex2Bundle inappropriate. However, the benefit of Ex2Bundle being LLM-free is that its latency scales with the corpus rather than model size, it applies to private data where LLM pipelines cannot run, and it incurs no per-query inference cost.

4.3.3 Comparison with LLM-based baselines as intermediate PaQL generator. We additionally compare against two LLM-based baselines that generate PaQL queries as the mechanism for snippet retrieval, rather than directly retrieving snippets. *LLM-agent* is an agent-based approach that performs the end-to-end pipeline of PaQL construction, bound relaxation, and query execution, while *LLM-Ex2Bundle* is a hybrid approach that delegates relaxation and execution to Ex2Bundle after the LLM generates the PaQL query. We evaluate both approaches on three SubSumE instances, allowing up to three self-critique-based relaxation steps per instance for LLM-agent. Table 11 summarizes the results.

Ex2Bundle outperforms both baselines across all metrics: (1) it is several orders of magnitude faster, (2) it retrieves more intent-aligned bundles, (3) it requires fewer relaxations, and (4) incurs no token cost. LLM-Ex2Bundle is 32× faster than LLM-agent and outperforms it in terms of quality metrics, runtime, and token cost, but requires more #relaxations. These results highlight two advantages of Ex2Bundle: superior initial bound synthesis and a data-aware relaxation strategy that outperforms LLMs' self-correction loop. Notably, LLM-Ex2Bundle's advantage over LLM-agent persists even when both start from LLM-generated bounds, demonstrating the benefit of Ex2Bundle's data-aware relaxation strategy. In contrast, LLM-generated bounds require more relaxation rounds, increasing not only runtime but also token cost.

- Ex2Bundle achieves the highest intent alignment across traditional and LLM-based baselines, especially for focused intents.
- LLM-generated PaQL queries struggle to produce intent-aligned bundles, while LLM-based bound relaxation is less effective than Ex2Bundle's data-aware relaxation.

4.4 Constraint-bundle alignment consistency

To evaluate RQ3, we test whether larger deviations from synthesized constraints produce larger deviations in the resulting bundles. For each query, we shift all topic constraints by the same additive amount s ($\mathsf{lb}' = \mathsf{lb} + s$, $\mathsf{ub}' = \mathsf{ub} + s$), using 15 log-spaced values in $[0.01, 20]$. We retain only feasible, unrelaxed original–shifted pairs, yielding approximately 411 pairs per topic (4,115 total); the resulting Hausdorff distances range from 0.01 to 11.3. We compute PCC between the Hausdorff distance of the induced feasible regions and the cosine distance between the corresponding bundle embeddings. Across topics, the distances are strongly correlated with an average PCC of 0.93, indicating that larger constraint deviations consistently produce larger bundle deviations.

- Bounds synthesized by Ex2Bundle consistently align with the intended bundles, achieving a PCC of 0.93.

4.5 Scalability analysis

We evaluate Ex2Bundle's scalability (RQ4) w.r.t (a) the size of the examples in #sentences; (b) the size of the target document in #sentences; and (c) the dimensionality of the documents in #topics. Ex2Bundle achieves practical scalability via the two mechanisms of §3.4, PreFilter and PaQL's SketchRefine. For small x, PreFilter keeps the ILP size tractable and independent of target document size. However, its effectiveness diminishes as x grows or limiting search to only $10x$ sentences becomes suboptimal, which is where SketchRefine contributes.

Over 400 instances of the SubSumE dataset, we vary the example size from 5 to 100 sentences and the target document size from 100K to 1M sentences, using 5 source and 45 target documents per instance. We increase the number of sentences in each document by

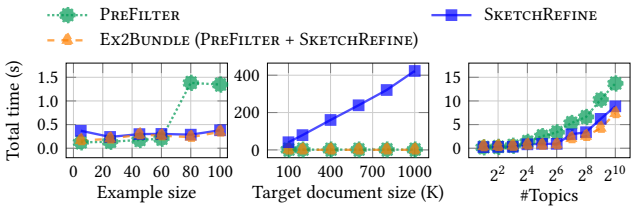


Figure 12: Ex2Bundle achieves scalability w.r.t example size (left), target document size (center), and topic dimensionality (right).

paraphrasing the original sentences using an LLM. To synthesize the example bundles, we iteratively sample sentences without replacement to avoid duplicates, randomly selecting sentences across topics until the required bundle size is reached.

Fig. 12 (left) shows that for small documents ($\approx$ 500 sentences), SketchRefine scales better than PreFilter as #sentences in the examples (x) grows. This is because PreFilter relies on direct ILP: when the document contains fewer than $10x$ sentences, it retains the full target document. In contrast, SketchRefine always considers the full document but reduces the search space via clustering and invokes ILP over cluster representatives. Ex2Bundle applies PreFilter before SketchRefine, benefiting from both. Fig. 12 (center) shows that for a fixed example size ($x = 100$), PreFilter's runtime increases linearly with target document size due to the single pass required to retain the top 1000 sentences, while SketchRefine's runtime increases more steeply. Ex2Bundle benefits from both and inherits the scalability of PreFilter, with runtime under 0.5 seconds even for documents with 1M sentences. Fig. 12 (right) shows that increasing document dimensionality in #topics (or constraints) causes linear runtime growth for all approaches (note x-axis is in log scale). However, practical workloads rarely exceed 1024 constraints, and even then, Ex2Bundle keeps its runtime under 9 seconds.

- Ex2Bundle scales nearly independently of document size.
- Ex2Bundle's runtime grows linearly with #topics, and remains under 9 seconds even for practically large #constraints.

4.6 Relaxation behavior and alternatives

4.6.1 Relaxation behavior analysis. We analyze relaxation behavior (RQ5) by varying the example size from 10 to 10K sentences and the target document size from 10 to 1M sentences. About 22% of queries are infeasible when varying example size and 47% when varying target document size. For infeasible queries, Fig. 13 (left) shows that #relaxations grows with example size, as more examples yield a higher lower bounds of the constraints, making them tighter. Conversely, #relaxations decreases with target document size, since larger documents are more likely to contain sentences satisfying the example-derived bounds (Fig. 13 (right)).

4.6.2 Alternative relaxation mechanisms. We contrast our symmetric relaxation (SYM), which relaxes both bounds of each violated constraint equally, against two alternatives: (1) *directional* (DIR) relaxation, which uses an Irreducible Infeasible Subsystem to determine each bound's relaxation direction, and (2) *FeasOpt* [36], which minimizes the total slack needed for feasibility. We force infeasibility using $pad \in \{30\%, 40\%, 50\%, 60\%\}$ with $(\mathtt{lb}, \mathtt{ub}) \mapsto ((1 + pad) \times \mathtt{lb}, (1 - pad) \times \mathtt{ub})$. Across 80 queries, these levels yield median relaxation rounds of 2/4/7/13 and infeasibility rates

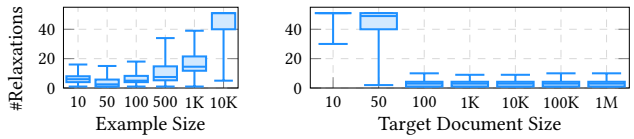


Figure 13: Number of relaxation rounds needed to reach feasibility as (left) example size and (right) target document size grow.

Rounds (n)	1–2 (8)	3–4 (9)	5–6 (21)	7–9 (18)	10–14 (22)	15+ (2)
SYM	**1.41**	2.16	1.38	1.86	8.59	5.83
DIR	1.48	1.98	**1.34**	1.88	8.64	5.39
FeasOpt	1.45	**1.27**	1.52	**1.53**	**1.84**	**1.33**

Table 14: Runtime comparison (in seconds) among Ex2Bundle's relaxation technique SYM, directional relaxation (DIR), & FeasOpt [36].

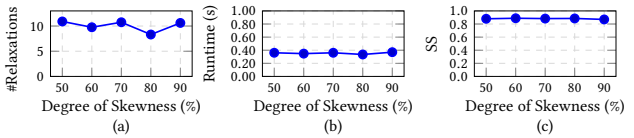


Figure 15: Ex2Bundle's (a) number of relaxations, (b) runtime, and (c) intent alignment as the topic skew in the target document varies.

of 66%/78%/97%/100%. Table 14 (bin size n) shows that FeasOpt outperforms the other methods in runtime mostly at high severity ($\geq$10 rounds). However, FeasOpt produces lower-quality bundles: in a separate evaluation over 177 queries, its quality score is lower than SYM's in 85% of cases and 8% lower on average. SYM matches DIR in 96.6% of cases and is 0.08% higher on average otherwise, since its relaxed feasible region is a superset of DIR's.

- More examples result in tighter constraints and require more #relaxation; larger documents provide more options to satisfy those bounds, requiring fewer #relaxations.
- The quality of SYM is never lower than DIR's, while FeasOpt's quality score is worse than SYM's in 85% of cases.

4.7 Robustness to data skew

To evaluate robustness to varying degrees of skewness in the target document's topic distribution (RQ6), we designate one topic as the *focus topic* and vary the fraction of sentences drawn from it (50%, 60%, 70%, 80%, and 90%), with the remaining sentences drawn uniformly from the other topics, over 100 instances of the SubSumE dataset. Fig. 15 shows the effect of increasing topic skew: (a) shows that the number of relaxations stays stable despite skew; (b) shows that runtime remains largely unaffected by target data skew; and (c) shows that the resulting bundles maintain a consistent level of alignment with the ground truth across all skew levels.

- Ex2Bundle's runtime, intent alignment, and #relaxations remain robust to skew in the target document's topic distribution.

Additional experimental results are in our technical report [14].

5 INTENT SLIDERS: A USER STUDY

To evaluate Ex2Bundle's slider-based intent-refinement interface for FTSE, we ran a within-subjects study with 20 Amazon Mechanical Turk participants (fluent in English, experienced with document search), who identified their preferred US state for relocation from Wikipedia articles using Ex2Bundle versus SBERT, the Top-k baseline (§4.3.1). We exclude non-BQbE baselines (e.g., conversational

LLMs), which underperform on FTSE (Table 10). We set the slider parameter $\alpha = 0.1$ and, following a mixed-methods approach [24], randomized each participant's starting tool (anonymized) to mitigate transfer effects. Each participant was assigned 25 randomly selected states per tool and asked to provide snippets for at least 2, with no upper limit. With Ex2Bundle, participants could select sentences, search by keyword, and refine intent via sliders; with SBERT, they could select sentences and search by keyword too, but had to adjust intent by manually adding or removing example sentences.

We quantitatively analyzed the data by comparing the participants' ratings of summary quality, ease of use, and preference for both SBERT and Ex2Bundle. We also compared the number of interactions with the interface (e.g., selecting examples, adjusting sliders, requesting snippet retrieval, etc.) between the two tools to understand how participants engaged with them. Participants' open-ended feedback on their experience additionally surfaced signals relevant to the perceived interpretability of the sliders. The majority used the sliders to refine their intents, and their comments suggest an interpretability benefit, for instance: *"I used them [sliders] to reduce importance of certain things and increase the importance of others to see how that would affect generated summaries"* and *"I raised the slider up so that the output summaries would prioritize information that pertained to the economy and climate of each state."*

Participants' satisfaction. On task completion, participants rated each tool on quality, ease of use, and daily usage preference. Ex2Bundle received higher ratings than SBERT on all three: (i) 12/20 rated Ex2Bundle as higher quality (8/20 for SBERT), (ii) 16/20 found Ex2Bundle easier to use (only 4/20 for SBERT), and (iii) 11/20 preferred Ex2Bundle for daily use (9/20 for SBERT).

Participants' interactions. To understand how participants interacted with the tools, we categorized interactions into six types: *Search*: keyword queries, *Selection*: example selection, *Update*: example modification, *Learn*: intent learning, *Retrieval*: backend retrieval, and *Slider*: interactive parameters. Averaging interaction counts per user, SBERT users performed about 3.1 times more *Search* interactions than Ex2Bundle users. In contrast, Ex2Bundle users had more *Slider* interactions and consequently about 2.1 times more *Retrieval* interactions, as Ex2Bundle automatically retrieves bundles upon slider adjustments. Ex2Bundle's interactive sliders encouraged more iterative refinement of results, while SBERT's lack of interactivity led to more manual searching.

6 RELATED WORK

Programming by example [27–29, 52] is a paradigm where users provide examples to express their intent and has seen significant success in data tasks such as data discovery by example [41, 55, 65] and query by example (QbE) [9, 16, 21, 22, 64, 71, 90]. QPlain [16] incorporates explanations with data provenance, some works prune the search space for efficient query synthesis [9, 71] or generalize the output [64], and recently SQuID [21, 22] semantically synthesizes SQL queries from user-provided examples rather than relying on structural similarity. However, all existing QbE systems are designed to retrieve individual items and do not account for the combinatorial nature of bundle retrieval. Ex2Bundle addresses the bundle retrieval problem through the lens of QbE, where users express their intent through example bundles, as opposed to example tuples.

Focused text snippet extraction, also known as query-focused extractive summarization, allows users to specify keywords or queries to guide snippet selection [39, 43, 45], but requires users to explicitly formulate their intents. RAG-based approaches [2, 31, 42, 46] can also support snippet extraction via few-shot prompting [10], but are not designed to jointly satisfy multiple constraints. *Faceted query-by-example* approaches leverage representative documents as an example query, additionally conditioned on explicit facets (constraints) to extract snippets across multiple constraints [17, 58, 80]. However, existing systems either require users to specify constraints via keywords [17] or natural language [80], or rely on opaque vector matching that lacks interpretability [58].

Bundle retrieval involves selecting an optimal set of items subject to a set of constraints [11, 13, 18, 51, 82]. It has also been studied extensively in recommender systems across various domains, such as e-commerce [47, 75, 84], entertainment [12, 62], and travel planning [19, 26, 88]. However, these systems rely on greedy heuristics or approximate generative models to generate bundles and struggle to enforce strict constraints. Package queries increase bundle query efficiency [11, 51], but require users to formulate the query. Diversification-based retrieval [13, 18, 82] retrieves bundles that collectively satisfy user constraints, but targets simpler constraints involving cardinality or diversity. In contrast, Ex2Bundle addresses the challenge of automatically synthesizing multi-constraint package queries from user examples.

Constraint relaxation. FeasOpt [36] relaxes infeasible constraints to obtain a feasible solution, potentially at the cost of solution quality. Related work in provenance and explanation addresses similar problems but in different settings. CAPE [54] explains surprising aggregate results by mining counterbalancing patterns from data outside the query's provenance, while QFix [81] diagnoses data errors by computing minimal repairs to historical queries that could cause the observed discrepancy. However, these approaches do not directly apply to our PaQL constraint relaxation problem.

7 SUMMARY AND FUTURE DIRECTIONS

We introduced Ex2Bundle, an example-driven framework for bundle retrieval that enables users to specify intents through example bundles. Ex2Bundle synthesizes package queries from examples and employs principled constraint relaxation to ensure feasibility. Our experiments and user study show that Ex2Bundle captures user intent and outperforms other baselines in intent alignment.

Several directions extend the framework. Manual example selection is burdensome, especially for complex intents or large data; an adaptive example recommendation that infers preference patterns from prior selections would lower this barrier. Ex2Bundle currently optimizes linear objectives over per-tuple scores; native PaQL support for pairwise or quadratic objectives (e.g., bundle diversity for playlists) would broaden expressiveness. Specification of the quality function requires domain expertise; inferring both objective and constraints from example bundles—a problem related to inverse optimization [34]—would remove the domain-expertise requirement. Choosing the best aggregate to use in the constraints is another open problem. LLMs can potentially suggest the best aggregate based on the task context, which will lower the configuration burden currently on the domain expert.

REFERENCES

[1] 2020. Discover Weekly keeps giving me the same genre that I'm completely sick of. https://community.spotify.com/t5/Your-Library/Discover-Weekly-keeps-giving-me-the-same-genre-that-I-m/td-p/5065068.

[2] 2026. LangChain. https://github.com/langchain-ai/langchain Accessed: 2026-01-07.

[3] 2026. Recommendations: Figuring out how to bring unique joy to each member. https://research.netflix.com/research-area/recommendations.

[4] Raghavendra Addanki, Andrew McGregor, Alexandra Meliou, and Zafeiria Moumoulidou. 2022. Improved Approximation and Scalability for Fair Max-Min Diversification. In *25th International Conference on Database Theory (ICDT 2022) (Leibniz International Proceedings in Informatics (LIPIcs))*, Vol. 220. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 7:1–7:21.

[5] Jinze Bai, Chang Zhou, Junshuai Song, Xiaoru Qu, Weiting An, Zhao Li, and Jun Gao. 2019. Personalized Bundle List Recommendation. In *The World Wide Web Conference, WWW 2019, San Francisco, CA, USA, May 13-17, 2019*. ACM, 60–71.

[6] Federico Bianchi, Silvia Terragni, Dirk Hovy, Debora Nozza, and Elisabetta Fersini. 2021. Cross-lingual Contextualized Topic Models with Zero-shot Learning. In *Proceedings of the 16th Conference of the European Chapter of the Association for Computational Linguistics: Main Volume*. Association for Computational Linguistics, Online, 1676–1683.

[7] David M. Blei, Andrew Y. Ng, and Michael I. Jordan. 2003. Latent Dirichlet Allocation. *J. Mach. Learn. Res.* 3 (2003), 993–1022.

[8] Jay Bonggolto. 2025. Google is testing a new 'Daily Discover' feed on YouTube Music. https://tech.yahoo.com/streaming/articles/google-testing-daily-discover-feed-170300058.html.

[9] Angela Bonifati, Radu Ciucanu, and Slawek Staworko. 2016. Learning Join Queries from User Examples. *ACM Trans. Database Syst.* 40, 4 (2016), 24:1–24:38.

[10] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, Christopher Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. 2020. Language Models are Few-Shot Learners. In *Advances in Neural Information Processing Systems 33: Annual Conference on Neural Information Processing Systems 2020, NeurIPS 2020, December 6-12, 2020, virtual*.

[11] Matteo Brucato, Juan Felipe Beltran, Azza Abouzied, and Alexandra Meliou. 2016. Scalable Package Queries in Relational Database Systems. *Proc. VLDB Endow.* 9, 7 (2016), 576–587.

[12] Jianxin Chang, Chen Gao, Xiangnan He, Depeng Jin, and Yong Li. 2023. Bundle Recommendation and Generation With Graph Neural Networks. *IEEE Trans. Knowl. Data Eng.* 35, 3 (2023), 2326–2340.

[13] Whanhee Cho and Anna Fariha. 2026. Data-Semantics-Aware Recommendation of Diverse Pivot Tables. *Proc. ACM Manag. Data* 4, 1, Article 23 (April 2026), 28 pages.

[14] Whanhee Cho, Kuangfei Long, Mahmood Jasim, Matteo Brucato, Alexandra Meliou, Peter J. Haas, and Anna Fariha. 2026. Example-Driven Intent Synthesis for Constrained Data Bundle Retrieval: Focused Text Snippet Extraction and Beyond. (2026). arXiv:2605.19246 [cs.DB] https://arxiv.org/abs/2605.19246

[15] John Joon Young Chung and Eytan Adar. 2023. PromptPaint: Steering Text-to-Image Generation Through Paint Medium-like Interactions. In *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology* (San Francisco, CA, USA) *(UIST '23)*. Association for Computing Machinery, New York, NY, USA, Article 6, 17 pages.

[16] Daniel Deutch and Amir Gilad. 2016. QPlain: Query by explanation. In *32nd IEEE International Conference on Data Engineering, ICDE 2016, Helsinki, Finland, May 16-20, 2016*. IEEE Computer Society, 1358–1361.

[17] Heejin Do, Sangwon Ryu, Jonghwi Kim, and Gary Lee. 2025. Multi-Facet Blending for Faceted Query-by-Example Retrieval. In *ACL*. 28577–28590.

[18] Marina Drosou and Evaggelia Pitoura. 2012. DisC diversity: result diversification based on dissimilarity and coverage. *Proc. VLDB Endow.* 6, 1 (2012), 13–24.

[19] Shih Hsin Fang, Eric Hsueh-Chan Lu, and Vincent S. Tseng. 2014. Trip Recommendation with Multiple User Constraints by Integrating Point-of-Interests and Travel Packages. In *IEEE 15th International Conference on Mobile Data Management, MDM 2014, Brisbane, Australia, July 14-18, 2014 - Volume 1*. IEEE Computer Society, 33–42.

[20] Anna Fariha, Matteo Brucato, Peter J. Haas, and Alexandra Meliou. 2020. SuDocu: Summarizing Documents by Example. *Proc. VLDB Endow.* 13, 12 (2020), 2861–2864.

[21] Anna Fariha, Lucy Cousins, Narges Mahyar, and Alexandra Meliou. 2026. Example-driven semantic-similarity-aware query intent discovery: Empowering users to cross the SQL barrier through query by example. *Inf. Syst.* 138 (2026), 102687.

[22] Anna Fariha and Alexandra Meliou. 2019. Example-Driven Query Intent Discovery: Abductive Reasoning using Semantic Similarity. *Proc. VLDB Endow.* 12, 11 (2019), 1262–1275.

[23] Rohit Gandikota, Joanna Materzynska, Tingrui Zhou, Antonio Torralba, and David Bau. 2024. Concept Sliders: LoRA Adaptors for Precise Control in Diffusion Models. In *Computer Vision - ECCV 2024 - 18th European Conference, Milan, Italy, September 29-October 4, 2024, Proceedings, Part XL (Lecture Notes in Computer Science)*. Springer, 172–188.

[24] Jennifer C Greene, Valerie J Caracelli, and Wendy F Graham. 1989. Toward a conceptual framework for mixed-method evaluation designs. *Educational evaluation and policy analysis* 11, 3 (1989), 255–274.

[25] Nianlong Gu, Elliott Ash, and Richard H. R. Hahnloser. 2022. MemSum: Extractive Summarization of Long Documents Using Multi-Step Episodic Markov Decision Processes. (2022), 6507–6522.

[26] Qi Gu, Jian Cao, and Yancen Liu. 2022. CSBR: A Compositional Semantics-Based Service Bundle Recommendation Approach for Mashup Development. *IEEE Trans. Serv. Comput.* 15, 6 (2022), 3170–3183.

[27] Sumit Gulwani. 2011. Automating string processing in spreadsheets using input-output examples. In *Proceedings of the 38th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, POPL 2011, Austin, TX, USA, January 26-28, 2011*. ACM, 317–330.

[28] Sumit Gulwani. 2016. Programming by Examples: Applications, Algorithms, and Ambiguity Resolution. In *Automated Reasoning - 8th International Joint Conference, IJCAR 2016, Coimbra, Portugal, June 27 - July 2, 2016, Proceedings (Lecture Notes in Computer Science)*, Vol. 9706. Springer, 9–14.

[29] Sumit Gulwani and Prateek Jain. 2017. Programming by Examples: PL Meets ML. In *Programming Languages and Systems - 15th Asian Symposium, APLAS 2017, Suzhou, China, November 27-29, 2017, Proceedings (Lecture Notes in Computer Science)*, Vol. 10695. Springer, 3–20.

[30] Wes Gurnee and Max Tegmark. 2024. Language Models Represent Space and Time. In *The Twelfth International Conference on Learning Representations, ICLR 2024, Vienna, Austria, May 7-11, 2024*. OpenReview.net.

[31] Kelvin Guu, Kenton Lee, Zora Tung, Panupong Pasupat, and Ming-Wei Chang. 2020. Retrieval Augmented Language Model Pre-Training. In *Proceedings of the 37th International Conference on Machine Learning, ICML 2020, 13-18 July 2020, Virtual Event (Proceedings of Machine Learning Research)*, Vol. 119. PMLR, 3929–3938.

[32] Karl Moritz Hermann, Tomás Kociský, Edward Grefenstette, Lasse Espeholt, Will Kay, Mustafa Suleyman, and Phil Blunsom. 2015. Teaching Machines to Read and Comprehend. In *Advances in Neural Information Processing Systems 28: Annual Conference on Neural Information Processing Systems 2015, December 7-12, 2015, Montreal, Quebec, Canada*. 1693–1701.

[33] Karl Moritz Hermann, Tomáš Kočiskỳ, Edward Grefenstette, Lasse Espeholt, Will Kay, Mustafa Suleyman, and Phil Blunsom. 2015. Teaching Machines to Read and Comprehend. In *Advances in Neural Information Processing Systems*, Vol. 28.

[34] Clemens Heuberger. 2004. Inverse Combinatorial Optimization: A Survey on Problems, Methods, and Results. *Journal of Combinatorial Optimization* 8, 3 (2004), 329–361.

[35] Andy Hunt, Marcelo M. Wanderley, and Matthew Paradis. 2002. The Importance of Parameter Mapping in Electronic Instrument Design. In *New Interfaces for Musical Expression, NIME-02, Proceedings, Dublin, Ireland, May 24-26, 2002*. Media Lab Europe, 149–154.

[36] IBM. 2015. *IBM ILOG CPLEX Optimization Studio: CPLEX User's Manual* (version 12.6 ed.).

[37] IBM ILOG CPLEX Optimization Studio [n.d.]. IBM ILOG CPLEX Optimization Studio. https://www.ibm.com/docs/en/icos.

[38] Stratos Idreos, Olga Papaemmanouil, and Surajit Chaudhuri. 2015. Overview of Data Exploration Techniques. In *SIGMOD*. ACM, 277–281.

[39] Hal Daumé III and Daniel Marcu. 2006. Bayesian Query-Focused Summarization. In *ACL 2006, 21st International Conference on Computational Linguistics and 44th Annual Meeting of the Association for Computational Linguistics, Proceedings of the Conference, Sydney, Australia, 17-21 July 2006*. The Association for Computer Linguistics.

[40] Rahul Jain, Amit Goel, Koichiro Niinuma, and Aakar Gupta. 2025. AdaptiveSliders: User-aligned Semantic Slider-based Editing of Text-to-Image Model Output. In *Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems (CHI '25)*. Association for Computing Machinery, New York, NY, USA, Article 541, 27 pages.

[41] Akash Khatri, Mahathir Mohammad, and El Kindi Rezig. 2025. Sort it Like You Mean It: Discovering Semantically Interesting Attribute Augmentations to Sort Tables. *Proc. VLDB Endow.* 18, 12 (2025), 5427–5430.

[42] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. 2020. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. In *NeurIPS*.

[43] Yanran Li and Sujian Li. 2014. Query-focused Multi-Document Summarization: Combining a Topic Model with Graph-based Semi-supervised Learning. In *Proceedings of COLING 2014, the 25th International Conference on Computational Linguistics: Technical Papers*. Dublin City University and Association for Computational Linguistics, Dublin, Ireland, 1197–1207.

[44] Chin-Yew Lin. 2004. Rouge: A package for automatic evaluation of summaries. In *Text summarization branches out*. 74–81.

[45] Marina Litvak and Natalia Vanetik. 2017. Query-based summarization using MDL principle. In *Proceedings of the multiling 2017 workshop on summarization and summary evaluation across source types and genres*. 22–31.

[46] Jerry Liu. 2022. *LlamaIndex*. https://github.com/jerryjliu/llama_index

[47] Xiaohao Liu, Jie Wu, Zhulin Tao, Yunshan Ma, Yinwei Wei, and Tat-Seng Chua. 2025. Fine-tuning Multimodal Large Language Models for Product Bundling. In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining, V.1, KDD 2025, Toronto, ON, Canada, August 3-7, 2025*. ACM, 848–858.

[48] Yang Liu and Mirella Lapata. 2019. Text Summarization with Pretrained Encoders. (2019), 3728–3738.

[49] Xuan Lu, Sifan Liu, Bochao Yin, Yongqi Li, Xinghao Chen, Hui Su, Yaohui Jin, Wenjun Zeng, and Xiaoyu Shen. 2025. MultiConIR: Towards Multi-Condition Information Retrieval. In *Findings of the Association for Computational Linguistics: EMNLP 2025*. Association for Computational Linguistics, Suzhou, China, 13471–13494.

[50] Gang Luo. 2006. Efficient Detection of Empty-Result Queries. In *Proceedings of the 32nd International Conference on Very Large Data Bases, Seoul, Korea, September 12-15, 2006*. ACM, 1015–1025.

[51] Anh L. Mai, Pengyu Wang, Azza Abouzied, Matteo Brucato, Peter J. Haas, and Alexandra Meliou. 2024. Scaling Package Queries to a Billion Tuples via Hierarchical Partitioning and Customized Optimization. *Proc. VLDB Endow.* 17, 5 (2024), 1146–1158.

[52] Mikaël Mayer, Gustavo Soares, Maxim Grechkin, Vu Le, Mark Marron, Oleksandr Polozov, Rishabh Singh, Benjamin G. Zorn, and Sumit Gulwani. 2015. User Interaction Models for Disambiguation in Programming by Example. In *UIST*. ACM, 291–301.

[53] Meta. 2024. Llama 3.3 70B Instruct FP8. https://huggingface.co/meta-llama/Llama-3.3-70B-Instruct.

[54] Zhengjie Miao, Qitian Zeng, Boris Glavic, and Sudeepa Roy. 2019. Going Beyond Provenance: Explaining Query Answers with Pattern-based Counterbalances. In *Proceedings of the 2019 International Conference on Management of Data, SIGMOD Conference 2019, Amsterdam, The Netherlands, June 30 - July 5, 2019*. ACM, 485–502.

[55] Mir Mahathir Mohammad and El Kindi Rezig. 2026. Qualitative Join Discovery in Data Lakes using Examples. *Proc. ACM Manag. Data* 4, 1, Article 68 (April 2026), 28 pages. https://doi.org/10.1145/3786682

[56] Sina Mohseni, Niloofar Zarei, and Eric D. Ragan. 2021. A Multidisciplinary Survey and Framework for Design and Evaluation of Explainable AI Systems. *ACM Trans. Interact. Intell. Syst.* 11, 3-4 (2021), 24:1–24:45.

[57] Christoph Molnar. 2020. *Interpretable machine learning*. Lulu. com.

[58] Sheshera Mysore, Arman Cohan, and Tom Hope. 2022. Multi-Vector Models with Textual Guidance for Fine-Grained Scientific Document Similarity. In *Proceedings of the 2022 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*. Association for Computational Linguistics, Seattle, United States, 4453–4470.

[59] Arnab Nandi and H. V. Jagadish. 2011. Guided Interaction: Rethinking the Query-Result Paradigm. *Proc. VLDB Endow.* 4, 12 (2011), 1466–1469.

[60] Ani Nenkova, Sameer Maskey, and Yang Liu. 2011. Automatic Summarization. In *The 49th Annual Meeting of the Association for Computational Linguistics: Human Language Technologies, Proceedings of the Conference, 19-24 June, 2011, Portland, Oregon, USA - Tutorial Abstracts*. The Association for Computer Linguistics, 3.

[61] OpenAI. 2023. GPT-4 Technical Report. arXiv:2303.08774 [cs.CL]

[62] Apurva Pathak, Kshitiz Gupta, and Julian J. McAuley. 2017. Generating and Personalizing Bundle Recommendations on *Steam*. In *SIGIR*. ACM, 1073–1076.

[63] Forough Poursabzi-Sangdeh, Daniel G. Goldstein, Jake M. Hofman, Jennifer Wortman Vaughan, and Hanna M. Wallach. 2021. Manipulating and Measuring Model Interpretability. In *CHI*. ACM, 237:1–237:52.

[64] Fotis Psallidas, Bolin Ding, Kaushik Chakrabarti, and Surajit Chaudhuri. 2015. S4: Top-k Spreadsheet-Style Search for Query Discovery. In *SIGMOD*. ACM, 2001–2016.

[65] El Kindi Rezig, Anshul Bhandari, Anna Fariha, Benjamin Price, Allan Vanterpool, Vijay Gadepally, and Michael Stonebraker. 2021. DICE: Data Discovery by Example. *Proc. VLDB Endow.* 14, 12 (2021), 2819–2822.

[66] R. Tyrrell Rockafellar and Roger J.-B. Wets. 1998. *Variational Analysis*. Springer, Berlin.

[67] Christian A. Scholbeck, Giuseppe Casalicchio, Christoph Molnar, Bernd Bischl, and Christian Heumann. 2024. Marginal effects for non-linear prediction functions. *Data Min. Knowl. Discov.* 38, 5 (2024), 2997–3042.

[68] Thibault Sellam and Martin L. Kersten. 2013. Meet Charles, big data query advisor. In *Sixth Biennial Conference on Innovative Data Systems Research, CIDR 2013, Asilomar, CA, USA, January 6-9, 2013, Online Proceedings*. www.cidrdb.org.

[69] Lesia Semenova, Cynthia Rudin, and Ronald Parr. 2022. On the Existence of Simpler Machine Learning Models. In *FAccT '22: 2022 ACM Conference on Fairness, Accountability, and Transparency, Seoul, Republic of Korea, June 21 - 24, 2022*. ACM, 1827–1858.

[70] Vidya Setlur, Sarah E. Battersby, Melanie Tory, Rich Gossweiler, and Angel X. Chang. 2016. Eviza: A Natural Language Interface for Visual Analysis. In *Proceedings of the 29th Annual Symposium on User Interface Software and Technology, UIST 2016, Tokyo, Japan, October 16-19, 2016*. ACM, 365–377.

[71] Yanyan Shen, Kaushik Chakrabarti, Surajit Chaudhuri, Bolin Ding, and Lev Novik. 2014. Discovering queries based on example tuples. In *SIGMOD*. ACM, 493–504.

[72] Yunxiao Shi, Haoning Shang, Xing Zi, Wujiang Xu, Yue Feng, and Min Xu. 2025. Answering Narrative-Driven Recommendation Queries via a Retrieve–Rank Paradigm and the OCG-Agent. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, Suzhou, China, 13181–13202.

[73] Spotify Advertising Team. 2020. Five years of discovery and engagement through Discover Weekly. https://ads.spotify.com/en-US/news-and-insights/five-years-of-discovery-and-engagement-through-discover-weekly/. Accessed: 2026-03-17.

[74] Hendrik Strobelt, Albert Webson, Victor Sanh, Benjamin Hoover, Johanna Beyer, Hanspeter Pfister, and Alexander M. Rush. 2023. Interactive and Visual Prompt Engineering for Ad-hoc Task Adaptation with Large Language Models. *IEEE Trans. Vis. Comput. Graph.* 29, 1 (2023), 1146–1156.

[75] Wenqi Sun, Ruobing Xie, Junjie Zhang, Wayne Xin Zhao, Leyu Lin, and Ji-Rong Wen. 2023. Generative Next-Basket Recommendation. In *Proceedings of the 17th ACM Conference on Recommender Systems, RecSys 2023, Singapore, Singapore, September 18-22, 2023*. ACM, 737–743.

[76] Nandan Thakur, Nils Reimers, Johannes Daxenberger, and Iryna Gurevych. 2021. Augmented SBERT: Data Augmentation Method for Improving Bi-Encoders for Pairwise Sentence Scoring Tasks. In *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*. Association for Computational Linguistics, Online, 296–310.

[77] Louis L. Thurstone. 1974. *A Law of Comparative Judgment* (1st ed.). Routledge. 12 pages.

[78] Transaction Processing Performance Council (TPC). [n.d.]. *TPC-H: Decision Support Benchmark*. Technical Report. TPC. http://www.tpc.org/tpch/

[79] Joseph Tso, Preston Schmittou, Quan Huynh, and Jibran Hutchins. 2026. ConstraintBench: Benchmarking LLM Constraint Reasoning on Direct Optimization. *arXiv preprint arXiv:2602.22465* (2026).

[80] Jianyou Wang, Kaicheng Wang, Xiaoyue Wang, Prudhviraj Naidu, Leon Bergen, and Ramamohan Paturi. 2023. DORIS-MAE: scientific document retrieval using multi-level aspect-based queries. , Article 1668 (2023), 16 pages.

[81] Xiaolan Wang, Alexandra Meliou, and Eugene Wu. 2017. QFix: Diagnosing Errors through Query Histories. In *Proceedings of the 2017 ACM International Conference on Management of Data, SIGMOD Conference 2017, Chicago, IL, USA, May 14-19, 2017*. ACM, 1369–1384.

[82] Yue Wang, Alexandra Meliou, and Gerome Miklau. 2018. RC-Index: Diversifying Answers to Range Queries. *Proc. VLDB Endow.* 11, 7 (2018), 773–786.

[83] Zhijie Wang, Yuheng Huang, Da Song, Lei Ma, and Tianyi Zhang. 2024. PromptCharm: Text-to-Image Generation through Multi-modal Prompting and Refinement. In *CHI*. ACM, 185:1–185:21.

[84] Penghui Wei, Shaoguo Liu, Xuanhua Yang, Liang Wang, and Bo Zheng. 2022. Towards Personalized Bundle Creative Generation with Contrastive Non-Autoregressive Decoding. In *SIGIR '22: The 45th International ACM SIGIR Conference on Research and Development in Information Retrieval, Madrid, Spain, July 11 - 15, 2022*. ACM, 2634–2638.

[85] Wen Xiao and Giuseppe Carenini. 2019. Extractive Summarization of Long Documents by Combining Global and Local Context. In *EMNLP-IJCNLP*. Association for Computational Linguistics, 3009–3019.

[86] Nishant Yadav, Matteo Brucato, Anna Fariha, Oscar Youngquist, Julian Killingback, Alexandra Meliou, and Peter Haas. 2021. SubSumE: A Dataset for Subjective Summary Extraction from Wikipedia Documents. In *New Frontiers in Summarization Workshop*. https://summarization2021.github.io

[87] Hamed Zamani, Bhaskar Mitra, Everest Chen, Gord Lueck, Fernando Diaz, Paul N. Bennett, Nick Craswell, and Susan T. Dumais. 2020. Analyzing and Learning from User Interactions for Search Clarification. In *SIGIR*. ACM, 1181–1190.

[88] Jiale Zhang, Mingqian Ma, Xiaofeng Gao, and Guihai Chen. 2024. Encoder-Decoder Based Route Generation Model for Flexible Travel Recommendation. *IEEE Trans. Serv. Comput.* 17, 3 (2024), 905–920.

[89] Siyan Zhao, Mingyi Hong, Yang Liu, Devamanyu Hazarika, and Kaixiang Lin. 2025. Do LLMs Recognize Your Preferences? Evaluating Personalized Preference Following in LLMs. In *ICLR*.

[90] Moshé M. Zloof. 1975. Query-by-Example: the Invocation and Definition of Tables and Forms. In *Proceedings of the International Conference on Very Large Data Bases, September 22-24, 1975, Framingham, Massachusetts, USA*. ACM, 1–24.